

Recurrent Neural Networks

Exercise



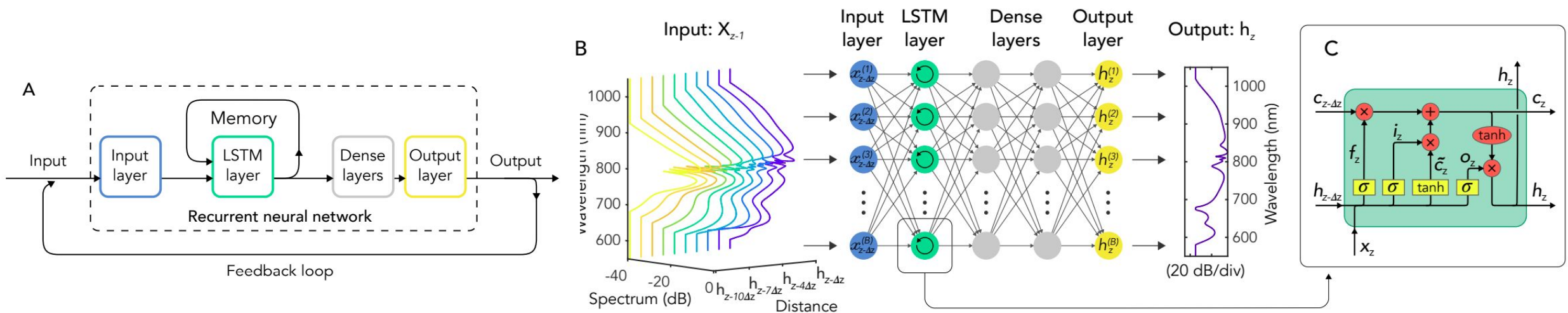
Nguyen Tho Anh Khoa

Nguyen Dang Nha

Duong Dinh Thang

Objectives

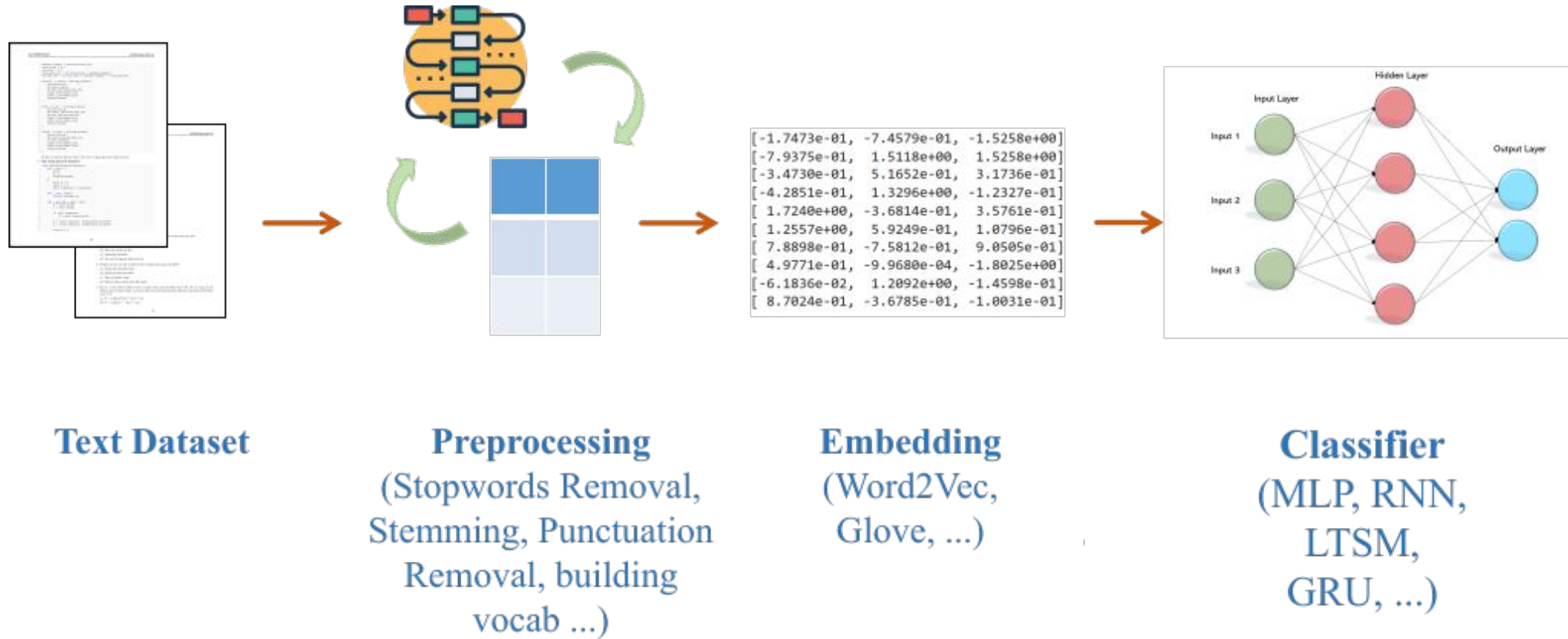
- Review RNN, LSTM and Bi-LSTM architecture.
- Apply RNN, LSTM, and Bi-LSTM to Financial News Sentiment Analysis
- Apply RNN, LSTM, and Bi-LSTM to Temperature Forecasting
- Recent Papers



Outline

- Review
- Financial News Sentiment Analysis
- Temperature Forecasting
- Question

❖ Text Classification Pipeline

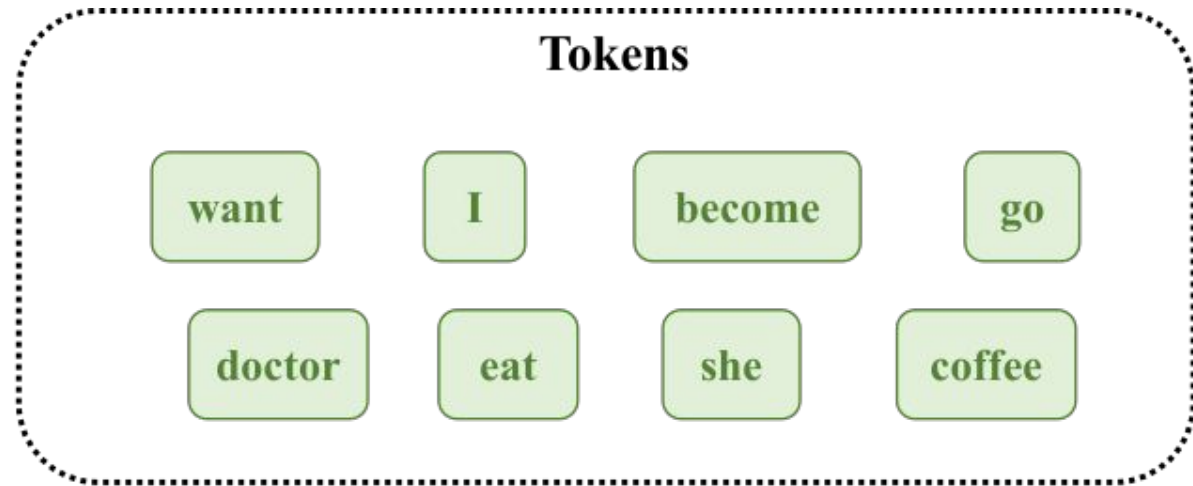


Review

❖ Preprocessing

Samples:

- 1: "I want to become a docter"
- 2: "She want to eat"
- 3: "Let's go to coffee"



Vocabulary

0	1	2	3	4	5	6	7	8	9
[UNK]	[PAD]	become	coffee	docter	eat	go	I	she	want

Review

❖ Embedding

Vocabulary

0	[UNK]
1	[PAD]
2	become
3	coffee
4	docter
5	eat
6	go
7	I
8	she
9	want

Embedding

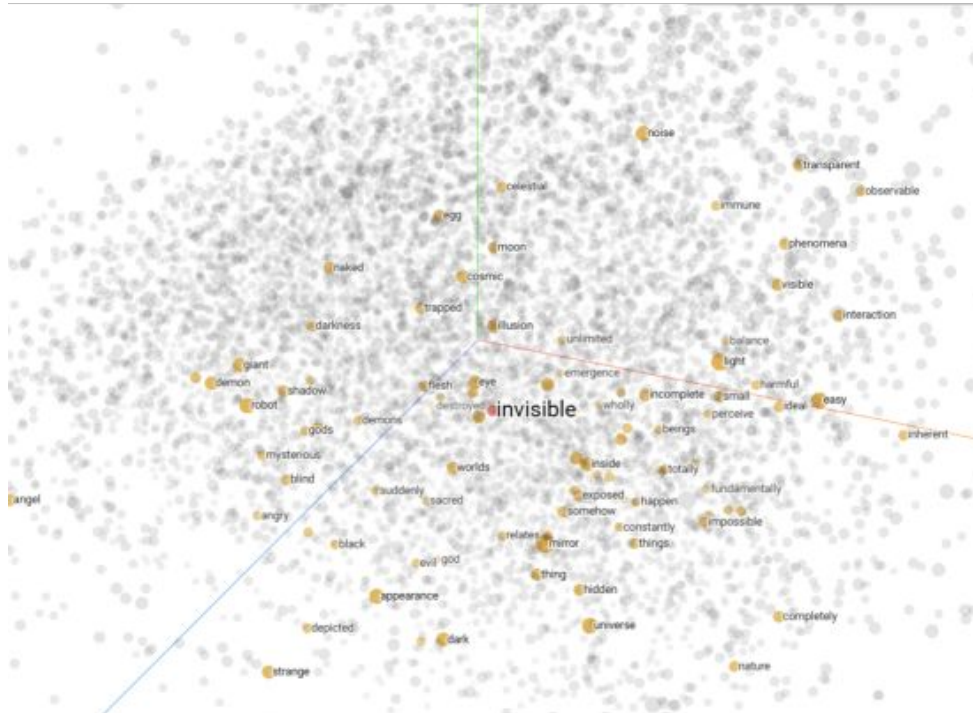
Word2Vec, GloVe, ...

Word Embedding

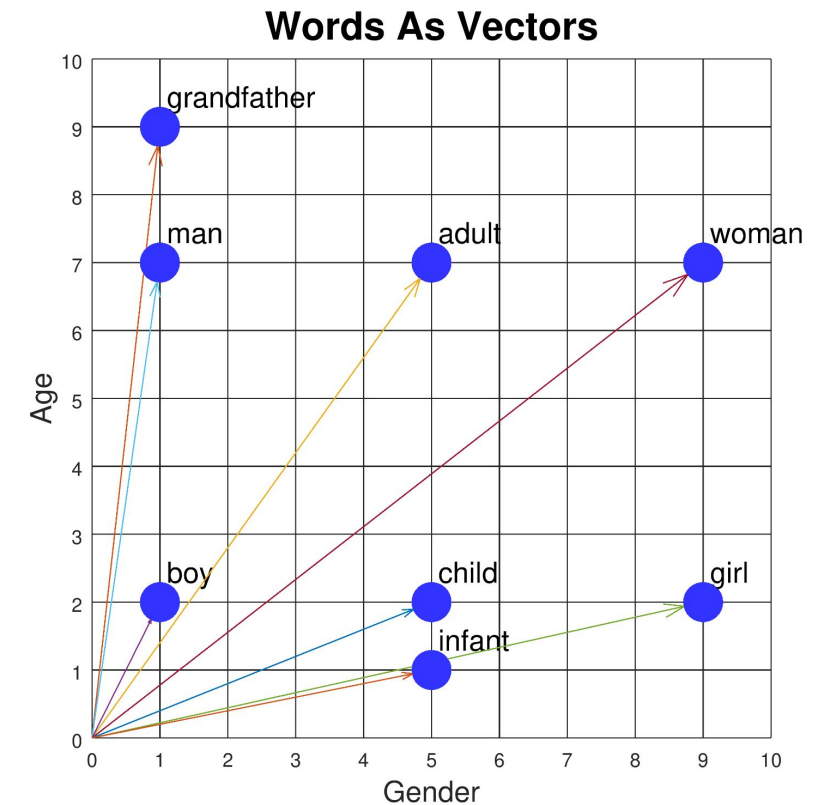
0	[0.37454012, 0.95071431, 0.73199394]
1	[0.59865848, 0.15601864, 0.15599452]
2	[0.05808361, 0.86617615, 0.60111501]
3	[0.70807258, 0.02058449, 0.96990985]
4	[0.83244264, 0.21233911, 0.18182497]
5	[0.18340451, 0.30424224, 0.52475643]
6	[0.43194502, 0.29122914, 0.61185289]
7	[0.13949386, 0.29214465, 0.36636184]
8	[0.45606998, 0.78517596, 0.19967378]
9	[0.51423444, 0.59241457, 0.04645041]

Dims = 3

◆ Embedding space



Example of Embedding Space
for large dataset



Review

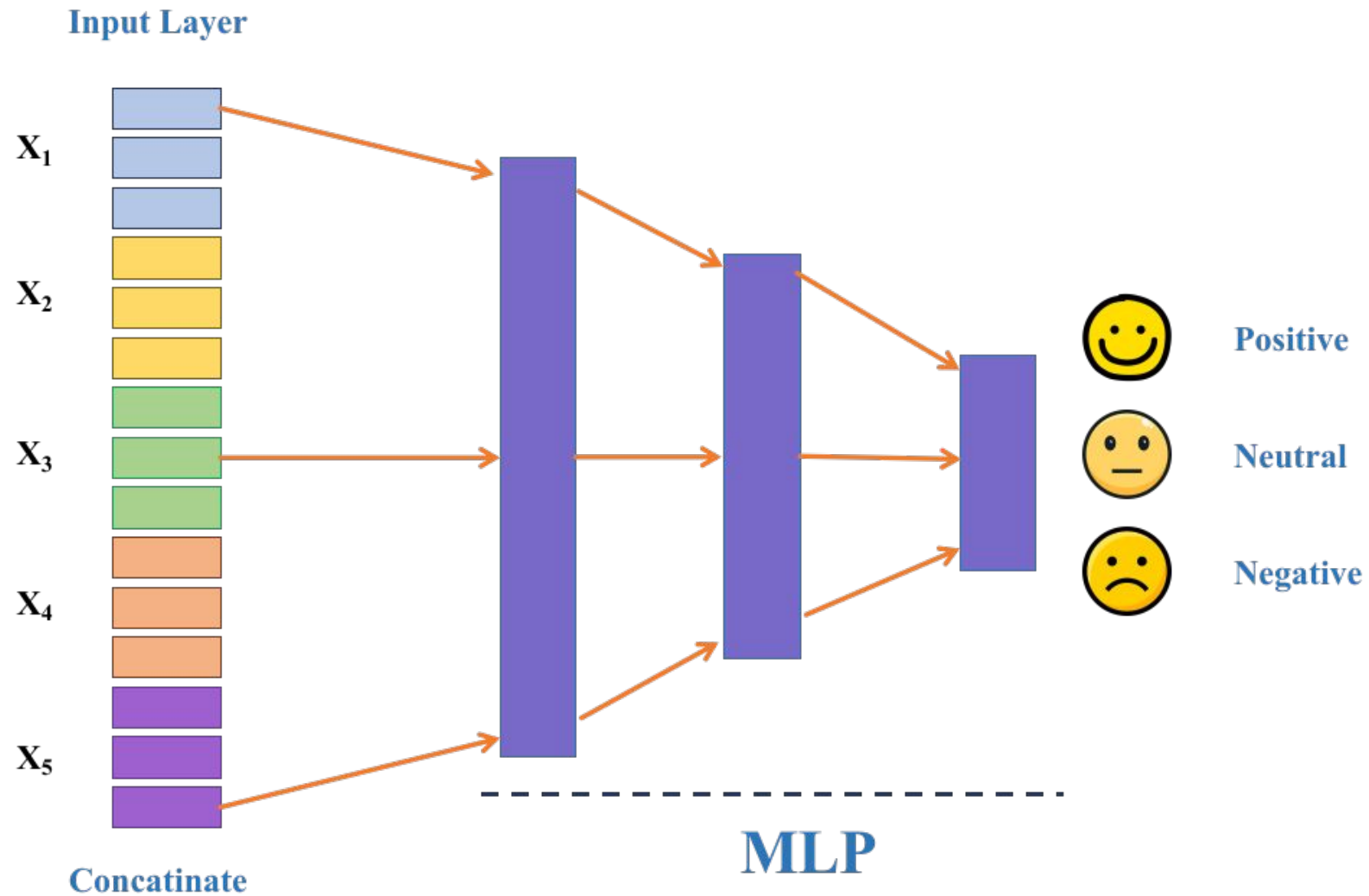
❖ Classifier

“Let’s go to coffee”

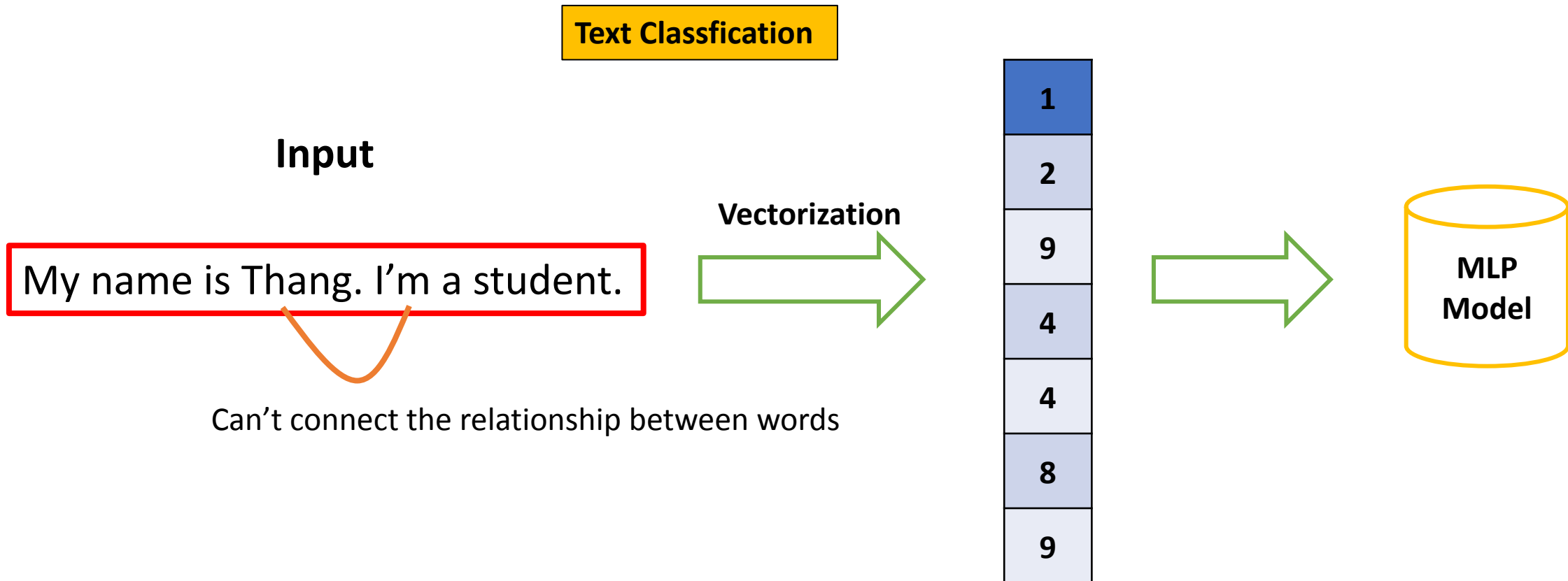


Review

❖ Classifier



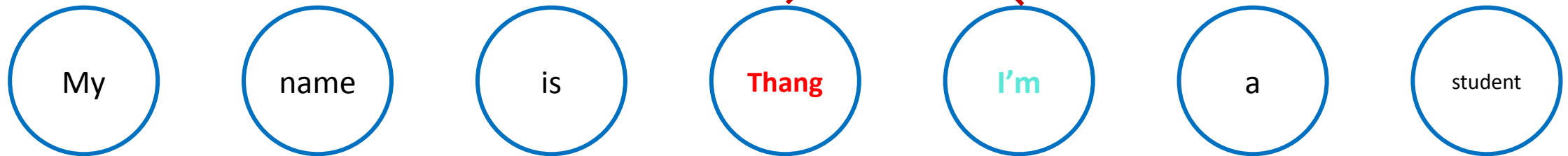
❖ Text Classification Problem



Review

❖ Basic Approach Problem

Can't connect the relationship between words

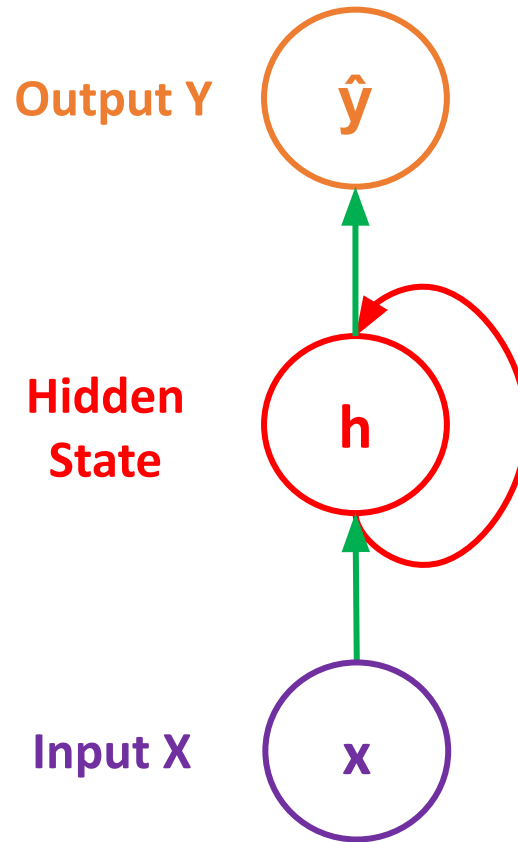


We want a vector that could represent all properties of a word.

Recurrent Neural Network

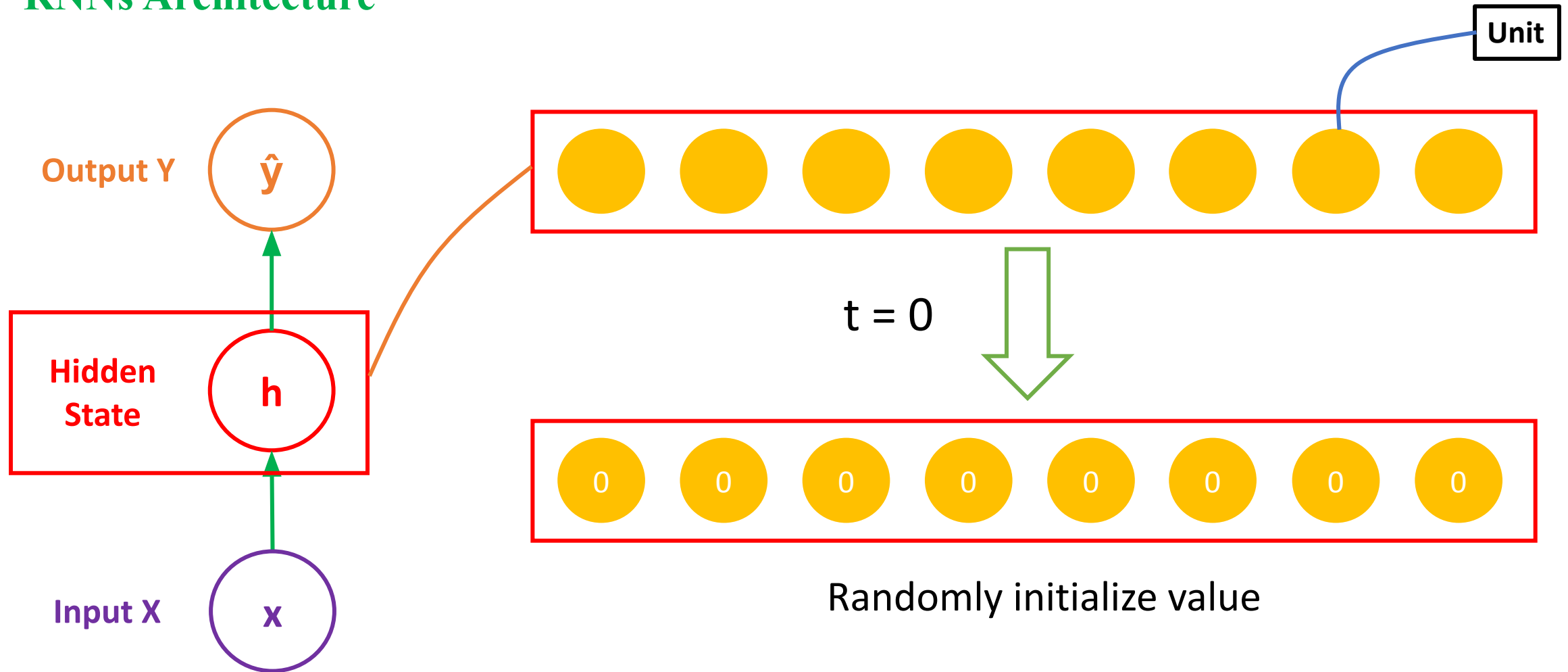
Review

◆ Recurrent Neural Networks (RNNs)



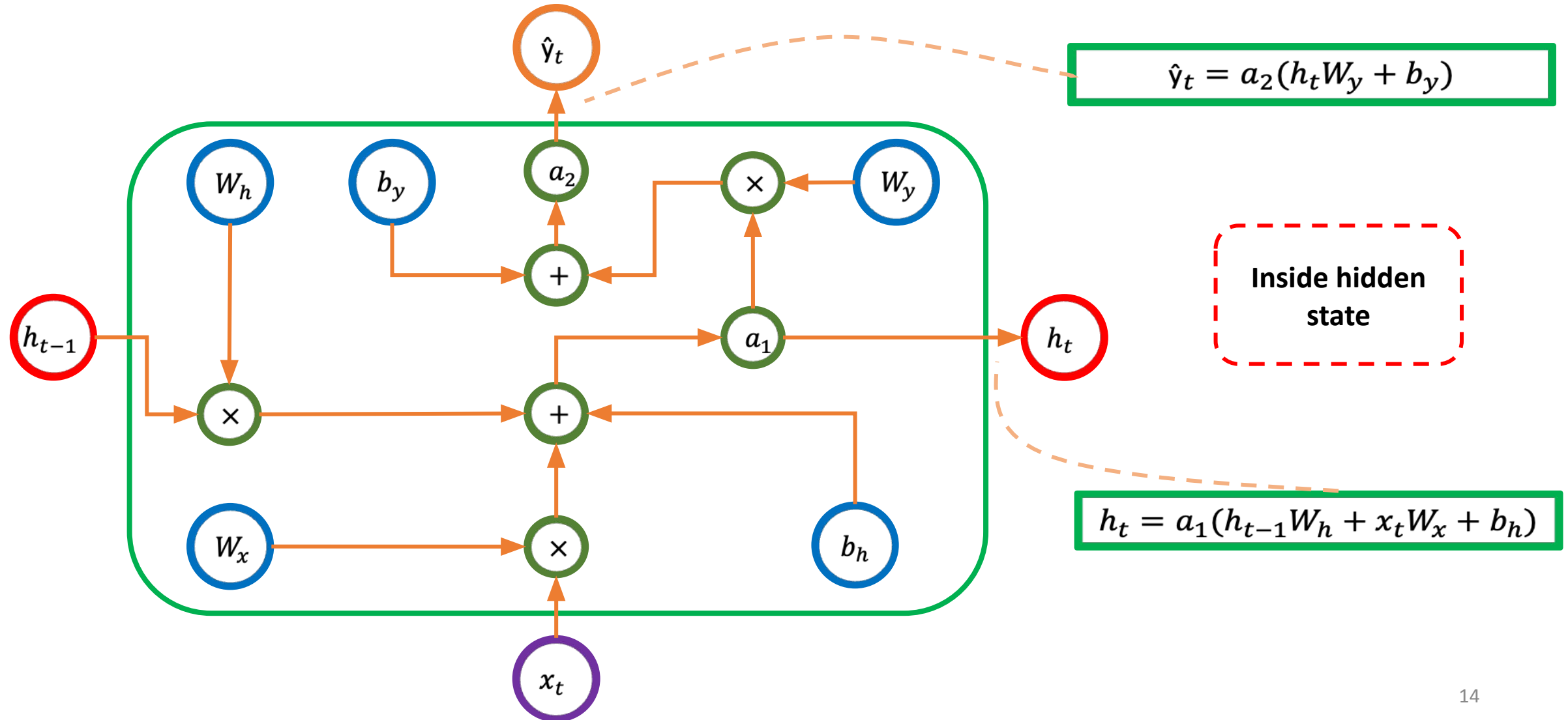
Review

❖ RNNs Architecture



Review

❖ RNNs Architecture



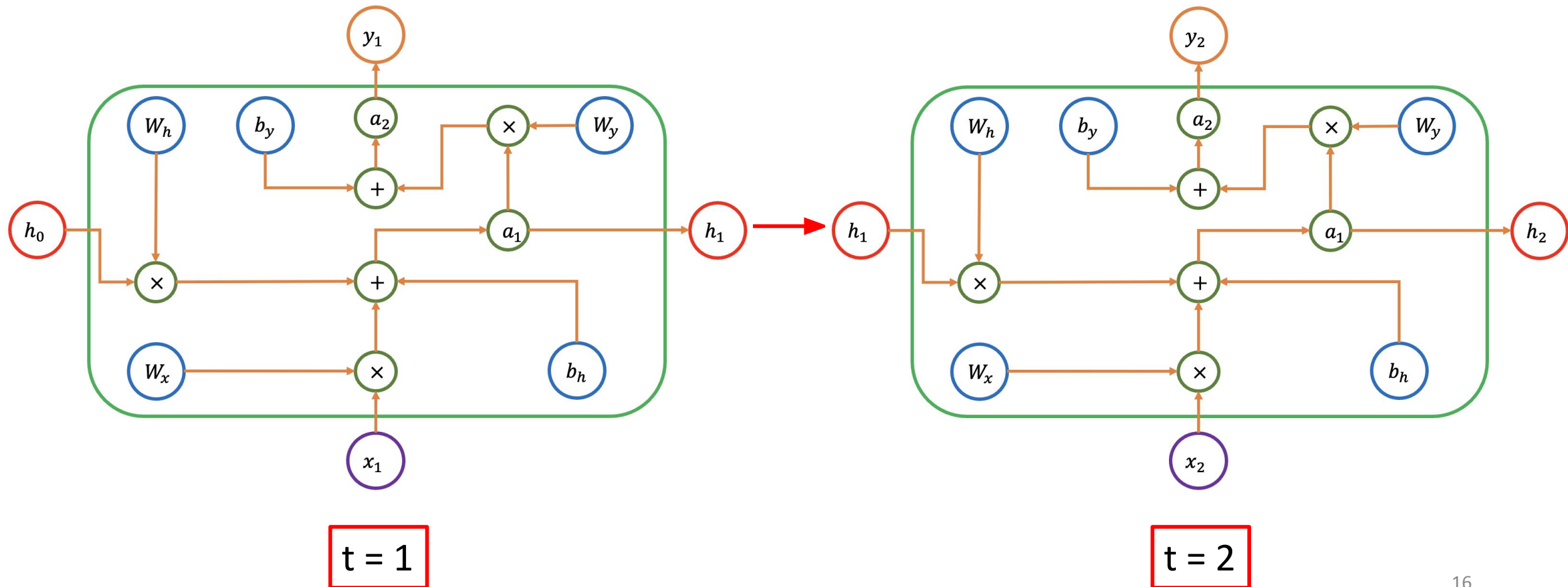
Quiz

Trong kiến trúc RNN, cách nào sau đây được sử dụng để duy trì thông tin qua các bước thời gian?

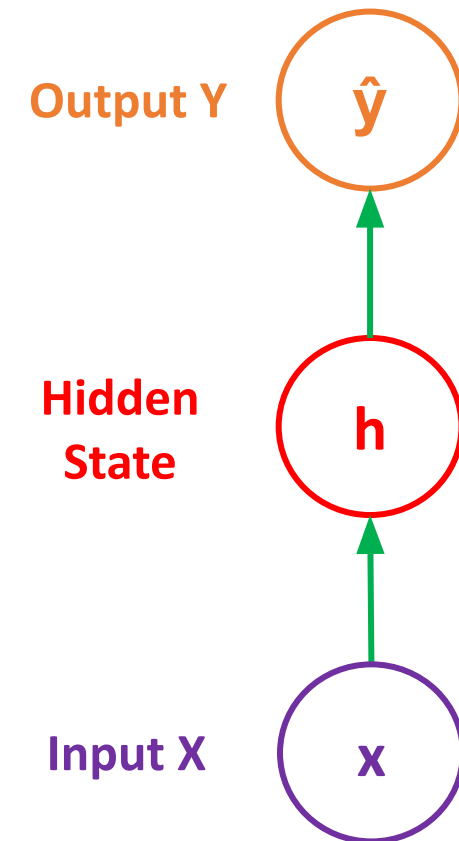
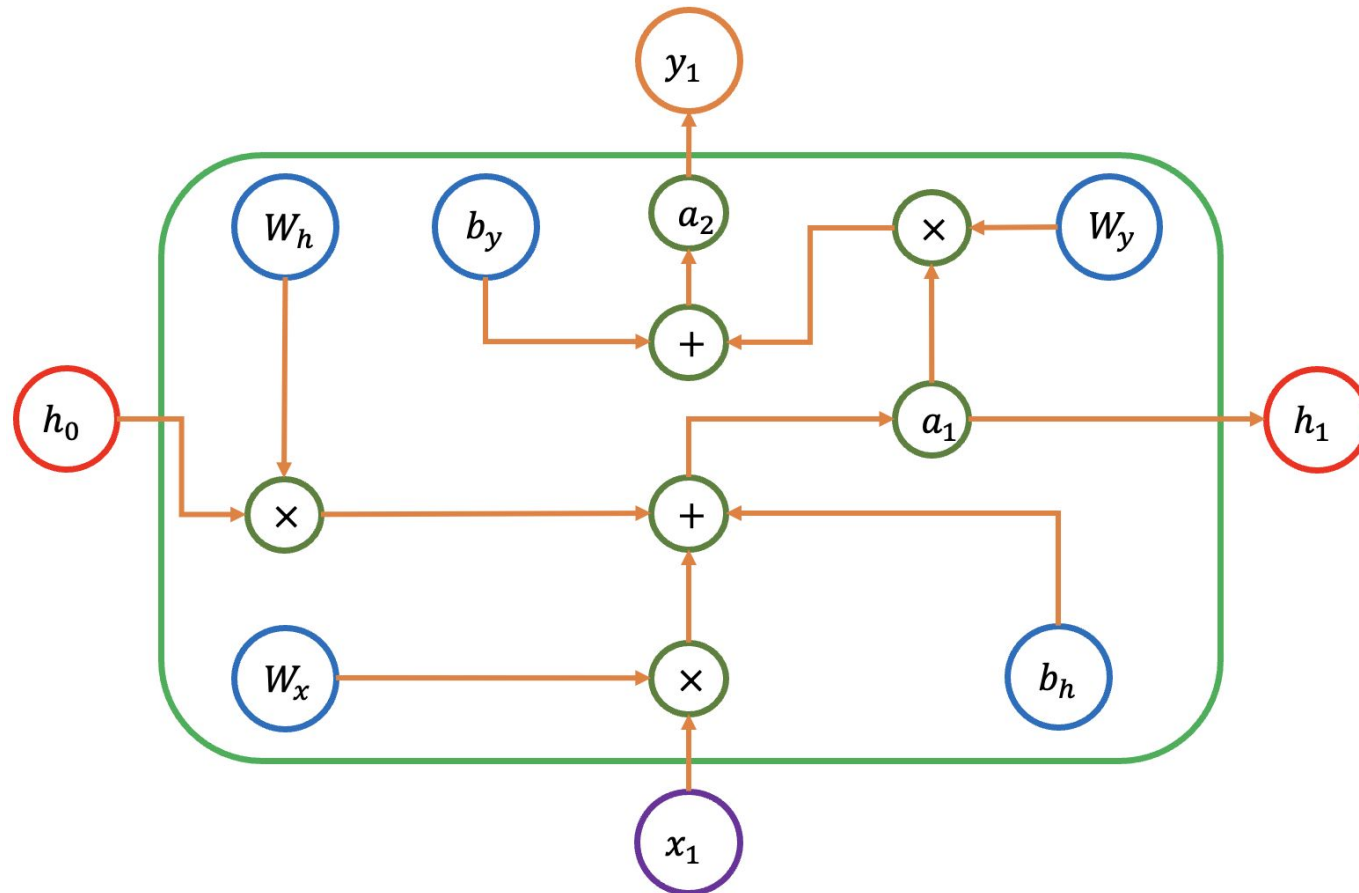
- a) Sử dụng mạng CNN**
- b) Sử dụng trạng thái ẩn (hidden state)**
- c) Sử dụng hàm kích hoạt ReLU**
- d) Sử dụng lớp đầu ra Softmax**

Review

❖ RNNs Architecture (from $t=1$ to $t=2$)

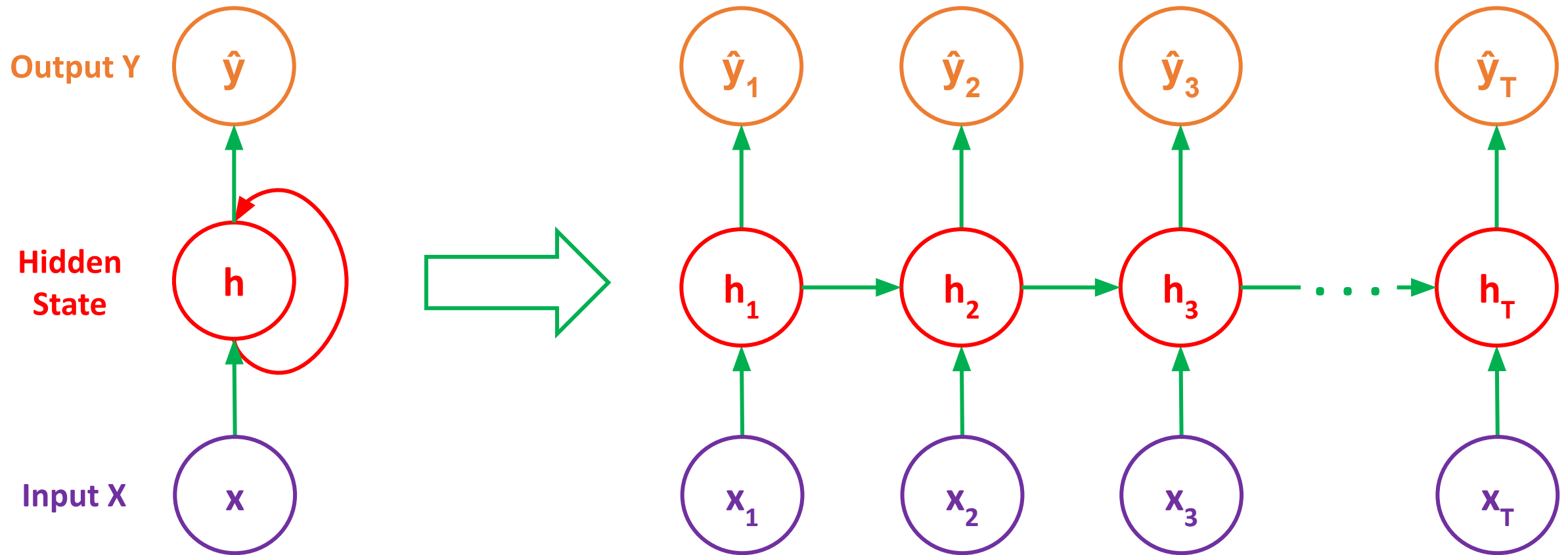


❖ RNNs Architecture (One-to-One)



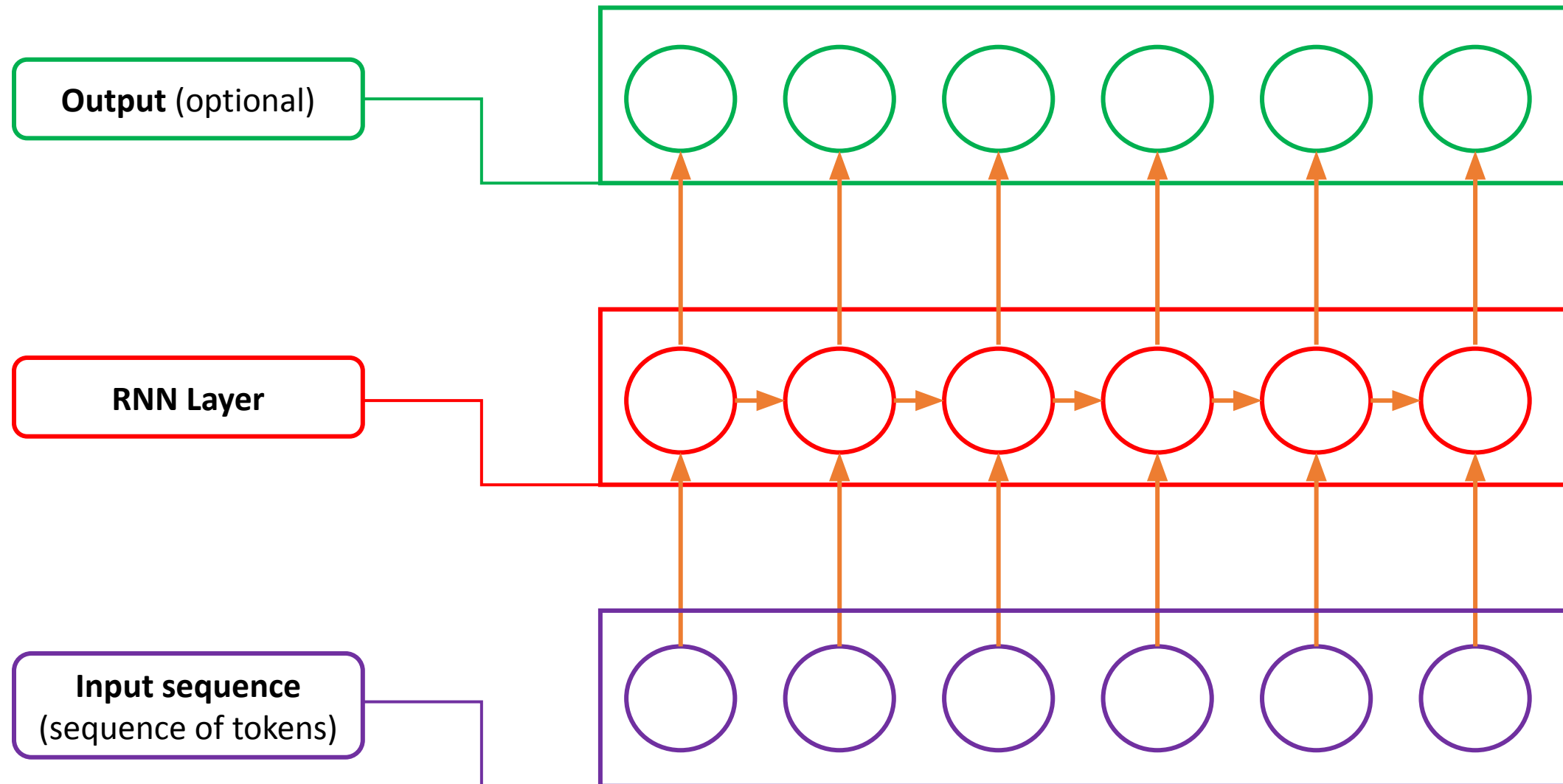
Review

◆ RNNs Architecture (Many-to-Many)



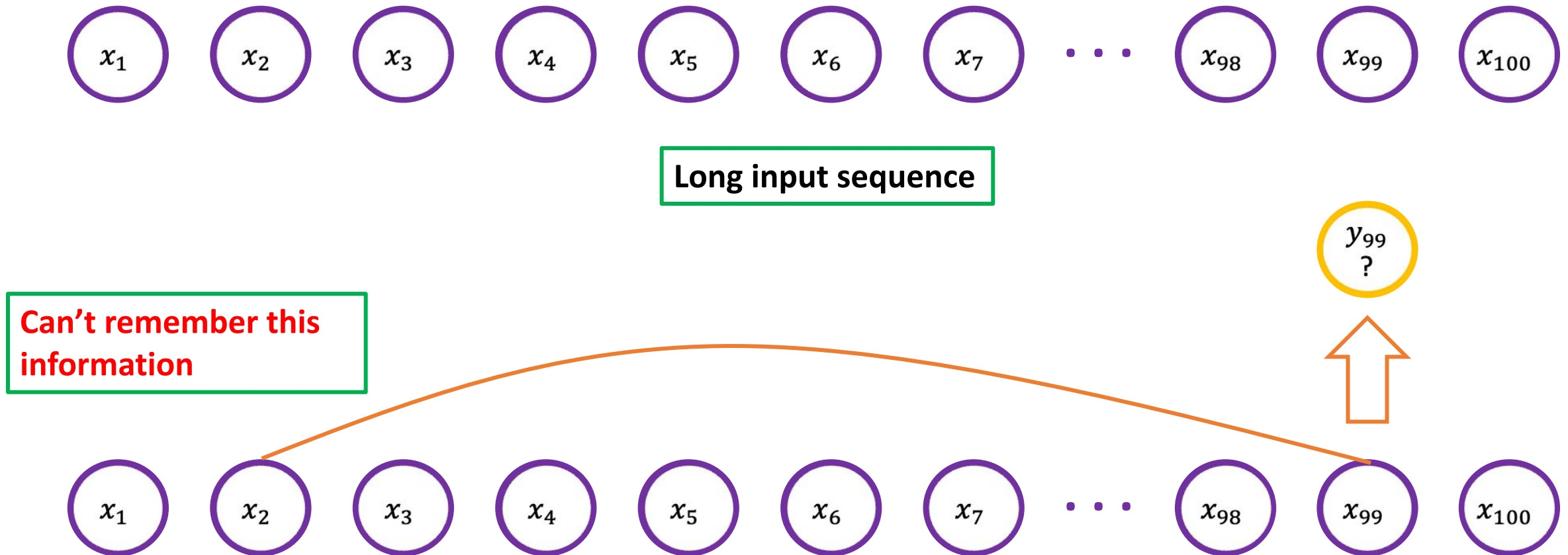
Review

❖ RNNs Architecture



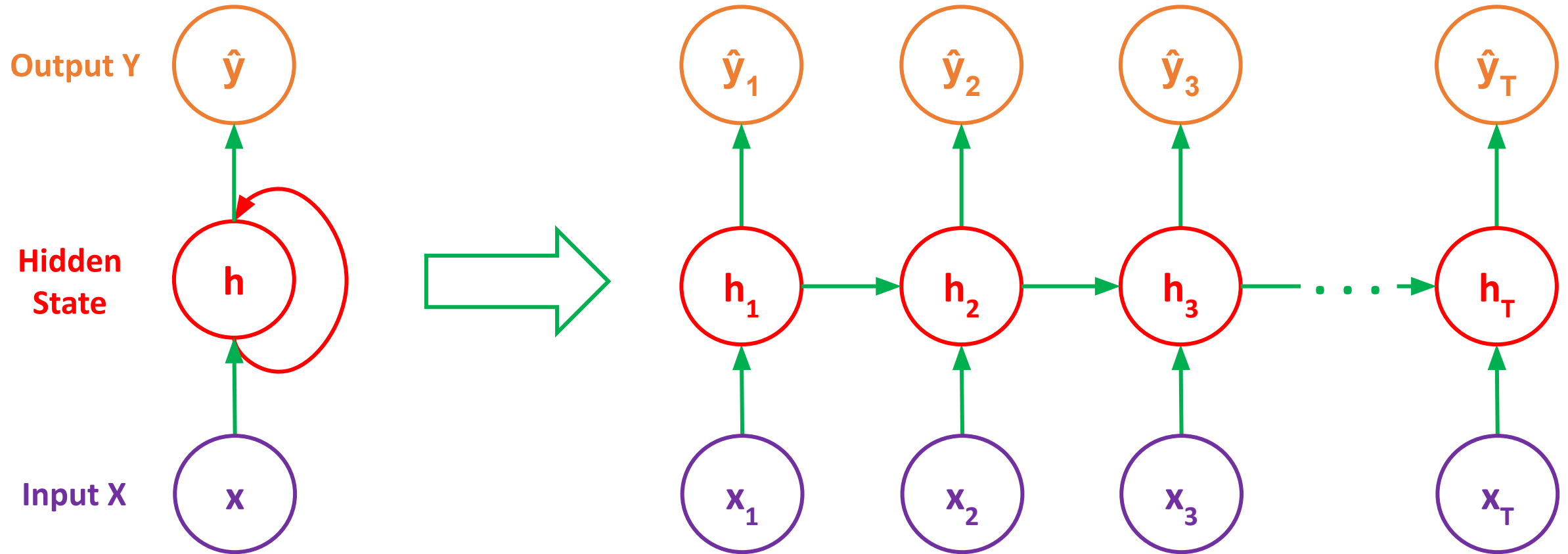
Review

❖ Long Short-Term Memory (LSTM)

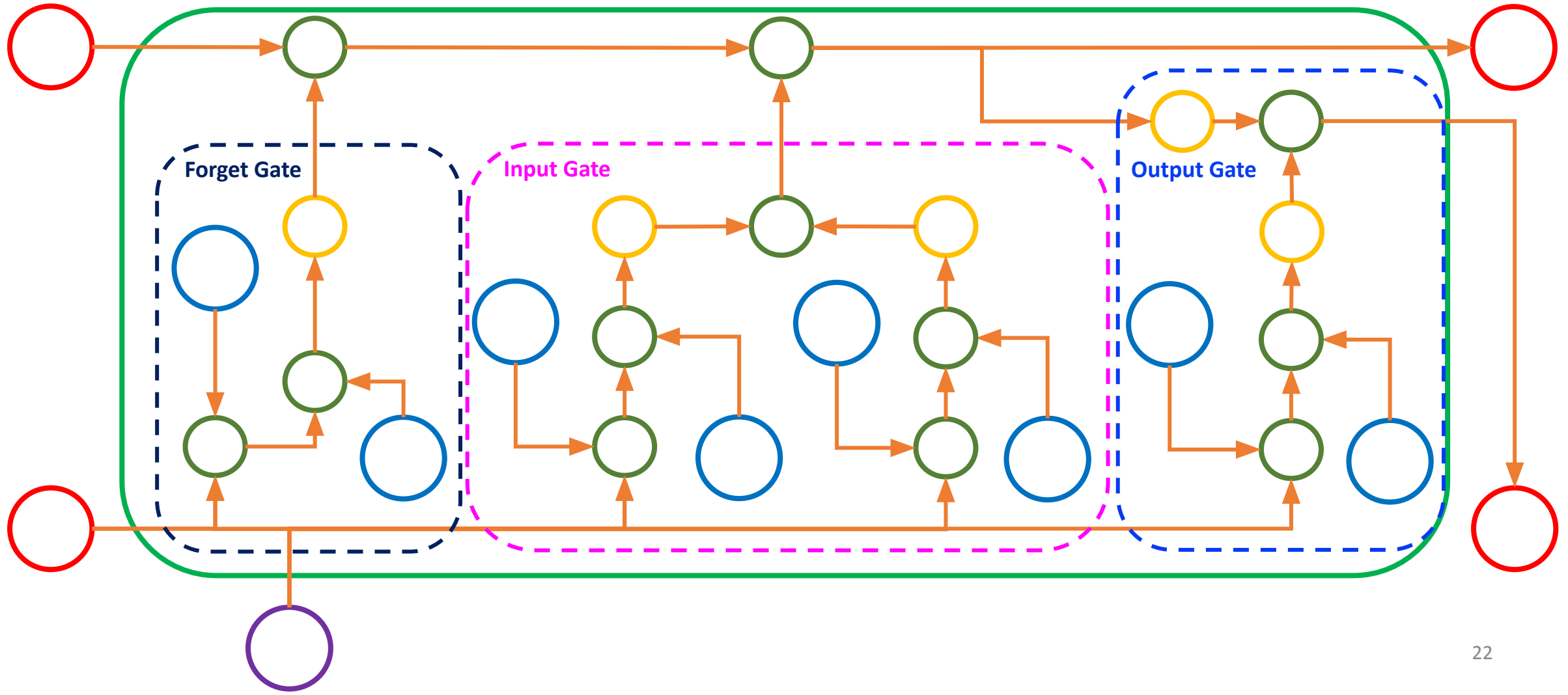


Review

❖ LSTM Architecture



❖ LSTM Architecture



❖ LSTM Architecture

Forget Gate:

- $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

Input Gate:

- $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$
- $C_t^i = \tanh(W_C[h_{t-1}, x_t] + b_C)$

Cell State:

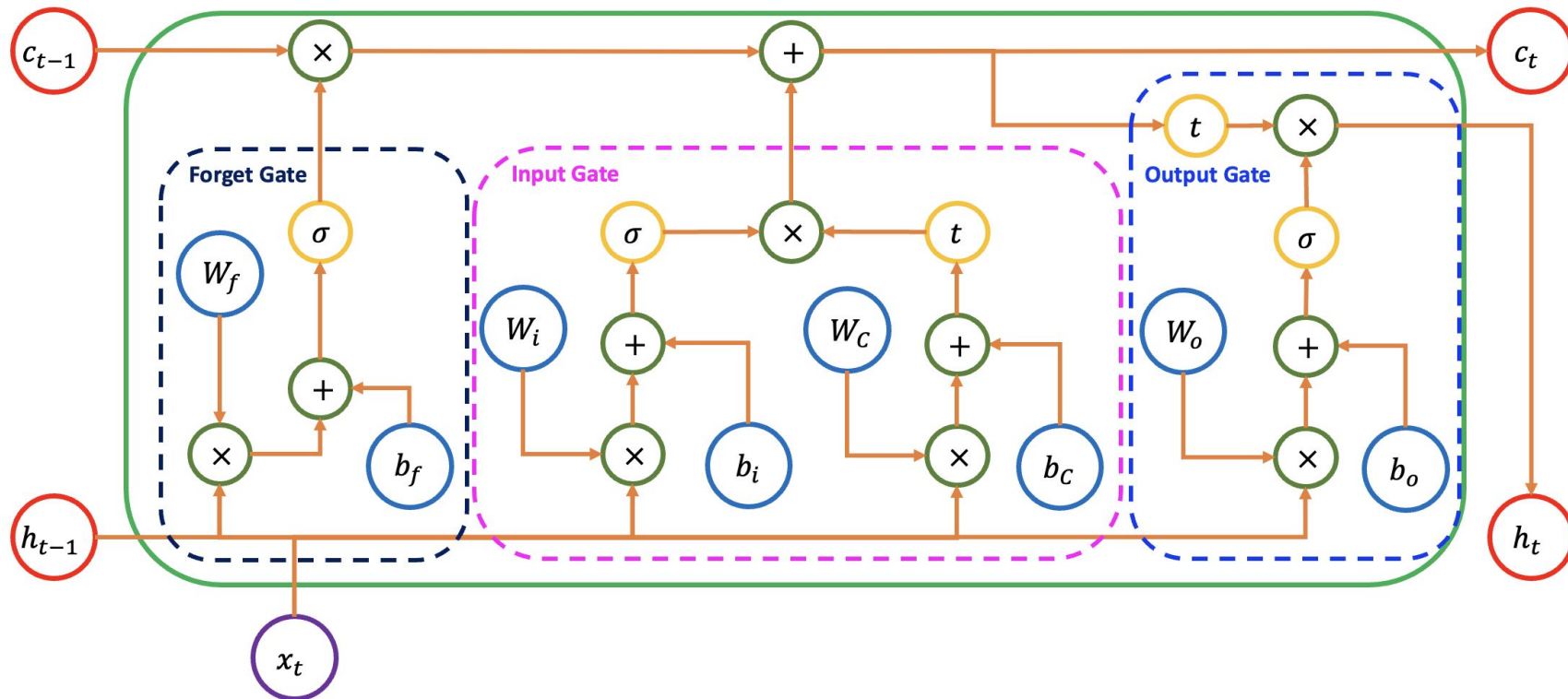
- $C_t = f_t C_{t-1} + i_t C_t^i$

Output Gate:

- $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$

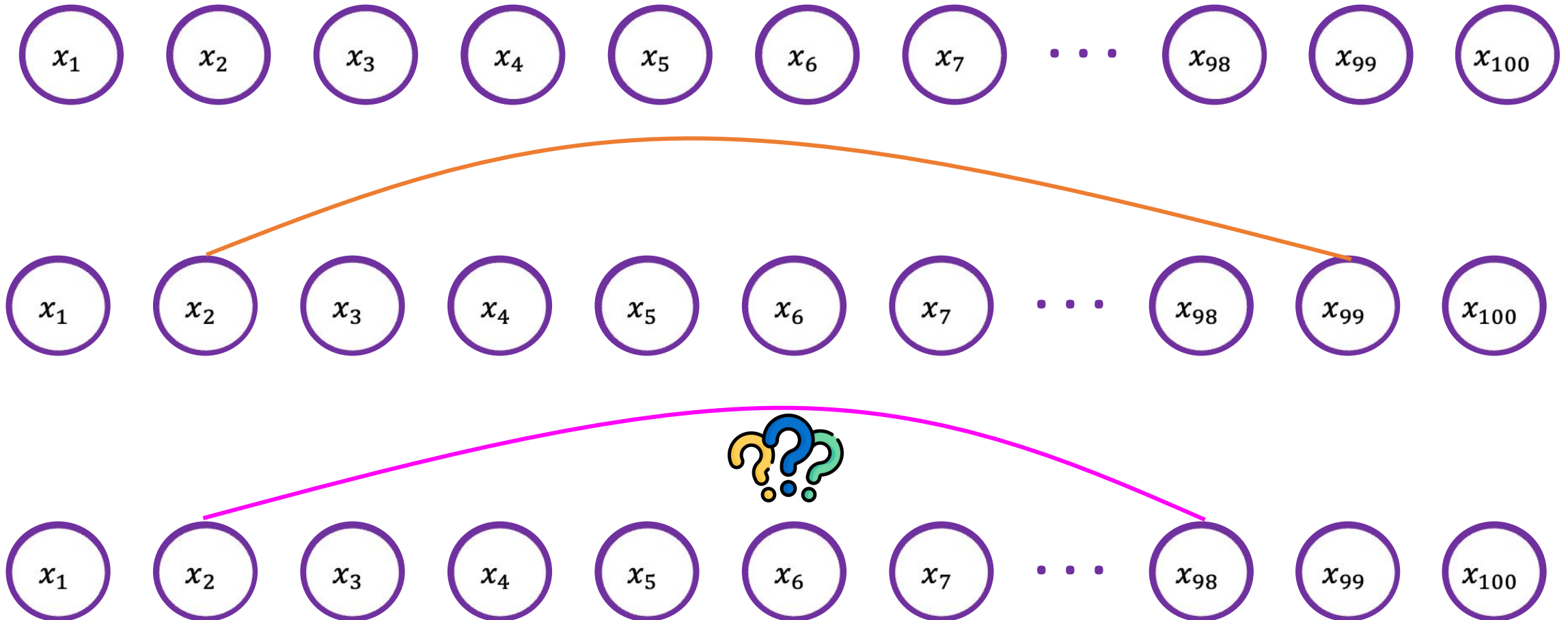
Hidden State:

- $h_t = \tanh(C_t) o_t$

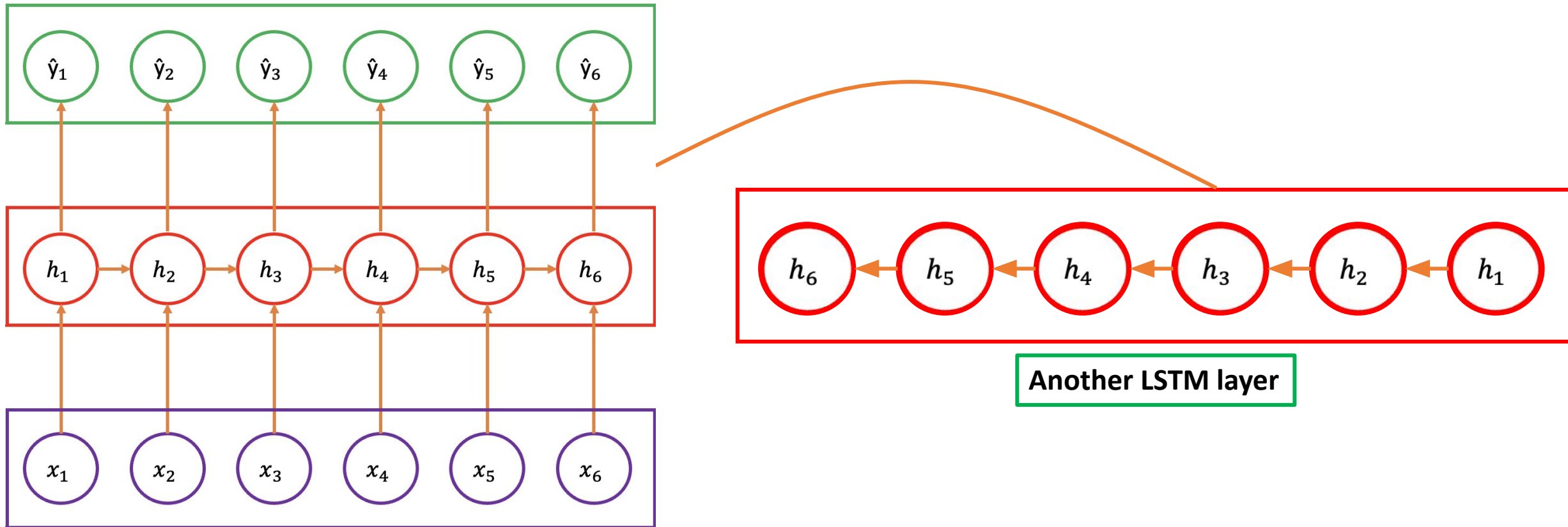


Review

◆ Bidirectional LSTM



◆ Bi-LSTM Architecture



Review

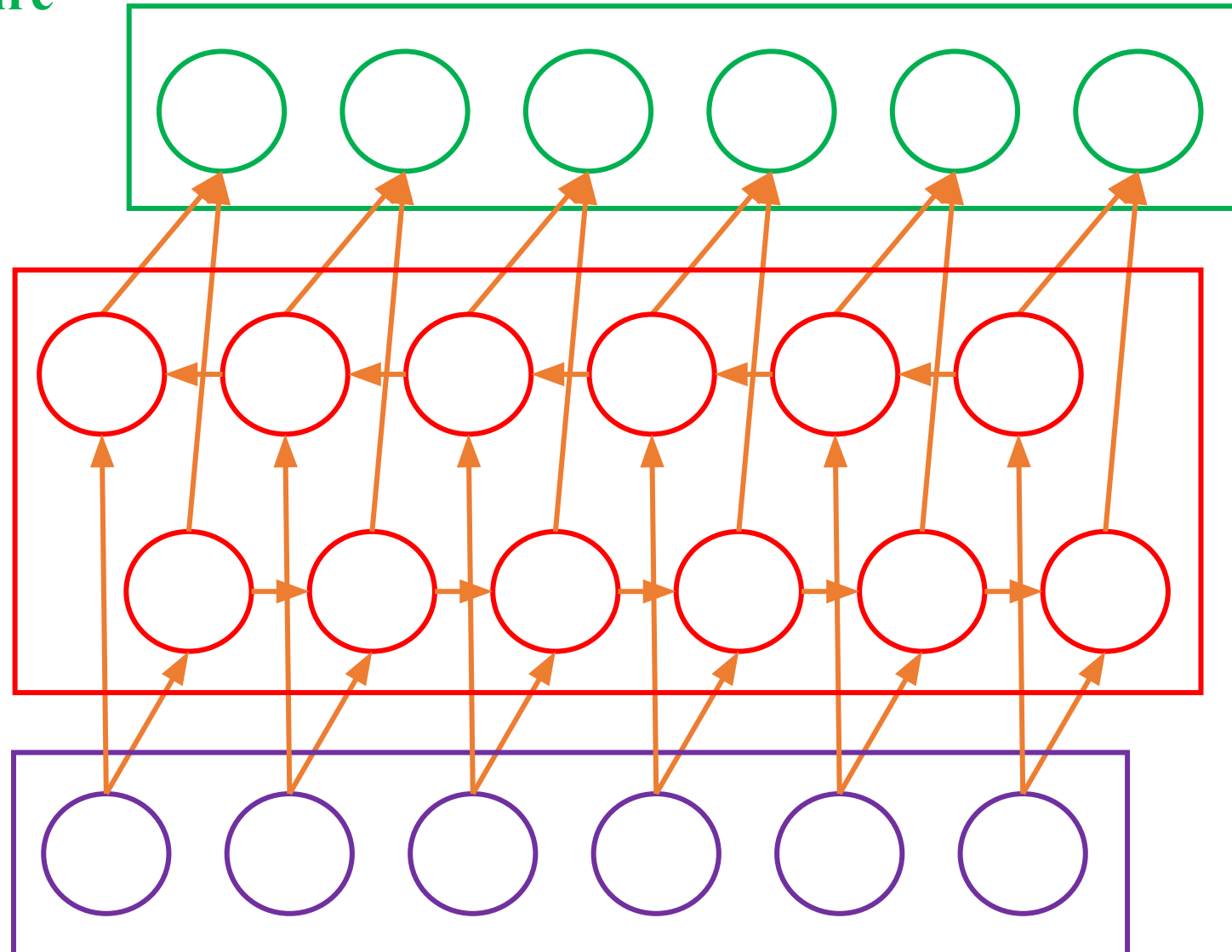
◆ Bi-LSTM Architecture

Output sequence

Backward LSTM

Forward LSTM

Input sequence



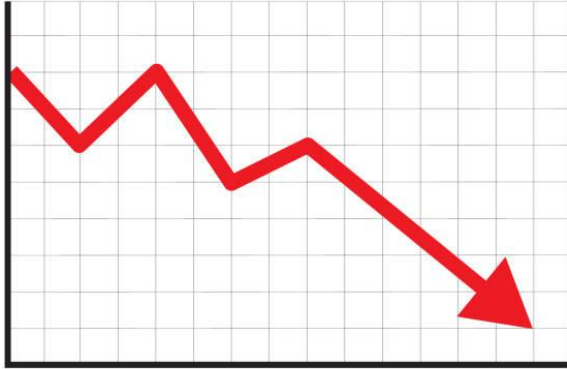
Quiz

Bi-directional LSTM (Bi-LSTM) khác với LSTM truyền thống như thế nào?

- a) Bi-LSTM chỉ xử lý chuỗi dữ liệu theo hướng thuận**
- b) Bi-LSTM xử lý chuỗi dữ liệu theo cả hai hướng thuận và ngược**
- c) Bi-LSTM không sử dụng cổng quên**
- d) Bi-LSTM có ít tham số hơn LSTM truyền thống**

Financial News Sentiment Analysis

❖ Coding Exercise



News: Dow closes more than 300 points lower as December selloff resumes.



Problem Statement: Given financial news dataset ([download here](#)), build a sentiment classifier using RNN/LSTM/BiLSTM layers.

Positive

Neutral

Negative

Financial News Sentiment Analysis

❖ Step 1: Import libraries

```
import torch
import torch.nn as nn

seed = 1
torch.manual_seed(seed)

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import re
import nltk
import unicode

nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer

from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
```



Financial News Sentiment Analysis

◆ Step 2: Read dataset

index	sentiment	content
0	neutral	According to Gran , the company has no plans to move all production to Russia , although that is where the company is growing .
1	neutral	Technopolis plans to develop in stages an area of no less than 100,000 square meters in order to host companies working in computer technologies and telecommunications , the statement said .
2	negative	The international electronic industry company Elcoteq has laid off tens of employees from its Tallinn facility ; contrary to earlier layoffs the company contracted the ranks of its office workers , the daily Postimees reported .
3	positive	With the new production plant the company would increase its capacity to meet the expected increase in demand and would improve the use of raw materials and therefore increase the production profitability .
4	positive	According to the company 's updated strategy for the years 2009-2012 , Basware targets a long-term net sales growth in the range of 20 % -40 % with an operating profit margin of 10 % -20 % of net sales .
5	positive	FINANCING OF ASPOCOMP 'S GROWTH Aspocomp is aggressively pursuing its growth strategy by increasingly focusing on technologically more demanding HDI printed circuit boards PCBs .
6	positive	For the last quarter of 2010 , Componenta 's net sales doubled to EUR131m from EUR76m for the same period a year earlier , while it moved to a zero pre-tax profit from a pre-tax loss of EUR7m .
7	positive	In the third quarter of 2010 , net sales increased by 5.2 % to EUR 205.5 mn , and operating profit by 34.9 % to EUR 23.5 mn .
8	positive	Operating profit rose to EUR 13.1 mn from EUR 8.7 mn in the corresponding period in 2007 representing 7.7 % of net sales .
9	positive	Operating profit totalled EUR 21.1 mn , up from EUR 18.6 mn in 2007 , representing 9.7 % of net sales .
10	positive	TeliaSonera TLSN said the offer is in line with its strategy to increase its ownership in core business holdings and would strengthen Eesti Telekom 's offering to its customers .
11	positive	STORA ENSO , NORSKE SKOG , M-REAL , UPM-KYMMENE Credit Suisse First Boston (CFSB) raised the fair value for shares in four of the largest Nordic forestry groups .
12	positive	A purchase agreement for 7,200 tons of gasoline with delivery at the Hamina terminal , Finland , was signed with Neste Oil OYJ at the average Platts index for this September plus eight US dollars per month .
13	positive	Finnish Talentum reports its operating profit increased to EUR 20.5 mn in 2005 from EUR 9.3 mn in 2004 , and net sales totalled EUR 103.3 mn , up from EUR 96.4 mn .
14	positive	Clothing retail chain Sepp+Æl+Æ 's sales increased by 8 % to EUR 155.2 mn , and operating profit rose to EUR 31.1 mn from EUR 17.1 mn in 2004 .
15	positive	Consolidated net sales increased 16 % to reach EUR74 .8 m , while operating profit amounted to EUR0 .9 m compared to a loss of EUR0 .7 m in the prior year period .

```
dataset_path = 'dataset/all-data.csv'
headers = ['sentiment', 'content']
df = pd.read_csv(
    dataset_path,
    names=headers,
    encoding='ISO-8859-1'
)
```

```
1 classes_lst = df['sentiment'].unique()
2 n_classes = len(classes_lst)
3
4 print(classes_lst)

['neutral' 'negative' 'positive']
```

Financial News Sentiment Analysis

❖ Step 2: Read dataset

```
classes = {  
    class_name: idx for idx, class_name \  
        in enumerate(df['sentiment'].unique().tolist())  
}  
df['sentiment'] = df['sentiment'].apply(lambda x: classes[x])  
df
```

1. Create a classes dictionary to map class name with its ID.
2. Apply ID mapping to the sentiment column.

	sentiment	content
0	0	According to Gran , the company has no plans t...
1	0	Technopolis plans to develop in stages an area...
2	1	The international electronic industry company ...
3	2	With the new production plant the company woul...
4	2	According to the company 's updated strategy f...
...	...	...
4841	1	LONDON MarketWatch -- Share prices ended lower...
4842	0	Rinkuskiai 's beer sales fell by 6.5 per cent ...
4843	1	Operating profit fell to EUR 35.4 mn from EUR ...
4844	1	Net sales of the Paper segment decreased to EU...
4845	1	Sales in Finland decreased by 10.5 % in Januar...

4846 rows × 2 columns

Financial News Sentiment Analysis

◆ Step 3: Text Normalization

Apply text normalization:

1. Lowercasing
2. Unidecoding
3. Stripping
4. Punctuation Removal
5. Stopwords Removal
6. Stemming

```
df['content'] =  
df['content'].apply(  
    lambda p:  
        text_normalize(p)).a  
        stype(str)
```

```
1 english_stop_words = stopwords.words('english')  
2 stemmer = PorterStemmer()  
3  
4 def text_normalize(text):  
5     text = text.lower()  
6     text = unidecode.unidecode(text)  
7     text = text.strip()  
8     text = re.sub(r'^\w\s', '', text)  
9     text = ' '.join([word for word in text.split(' ') if word not in english_stop_words])  
10    text = ' '.join([stemmer.stem(word) for word in text.split(' ')])  
11  
12    return text
```

According to the company's updated strategy for the years 2009-2012 ,
Basware targets a long-term net sales growth in the range of 20 % -40 %
with an operating profit margin of 10 % -20 % of net sales .

accord compani updat strategi year 20092012 baswar target longterm net
sale growth rang 20 40 oper profit margin 10 20 net sale

Financial News Sentiment Analysis

❖ Step 4: Build transform function

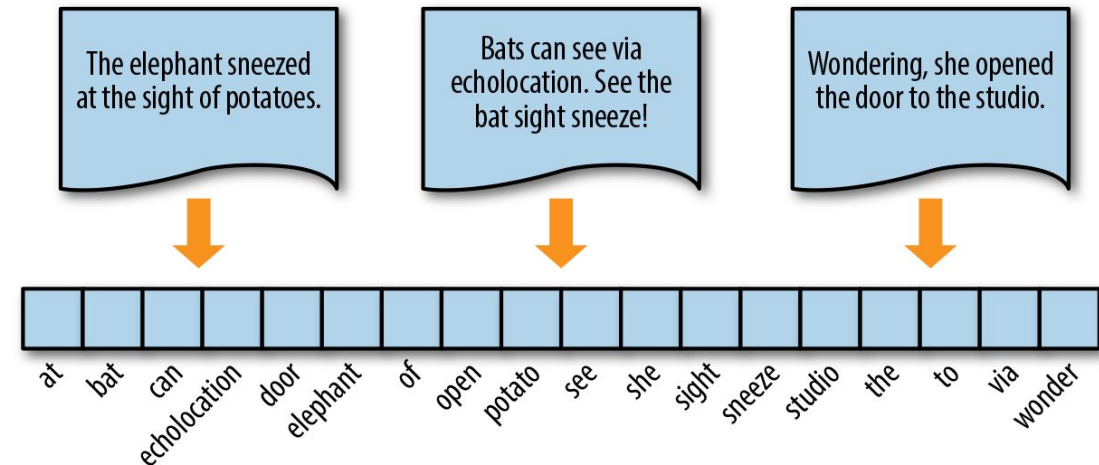
```
vocab = []
for sentence in df['content'].tolist():
    tokens = sentence.split()
    for token in tokens:
        if token not in vocab:
            vocab.append(token)

vocab.append('UNK')
vocab.append('PAD')
word_to_idx = {word: idx for idx, word in enumerate(vocab)}
vocab_size = len(vocab)

def transform(text, word_to_idx, max_seq_len):
    tokens = []
    for w in text.split():
        try:
            w_ids = word_to_idx[w]
        except:
            w_ids = word_to_idx['UNK']
        tokens.append(w_ids)

    if len(tokens) < max_seq_len:
        tokens += [word_to_idx['PAD']] * (max_seq_len - len(tokens))
    elif len(tokens) > max_seq_len:
        tokens = tokens[:max_seq_len]

    return tokens
```



Financial News Sentiment Analysis

❖ Step 5: Train, val, test split

```
val_size = 0.2
test_size = 0.125
is_shuffle = True
texts = df['content'].tolist()
labels = df['sentiment'].tolist()

X_train, X_val, y_train, y_val = train_test_split(
    texts, labels,
    test_size=val_size,
    random_state=seed,
    shuffle=is_shuffle
)

X_train, X_test, y_train, y_test = train_test_split(
    X_train, y_train,
    test_size=test_size,
    random_state=seed,
    shuffle=is_shuffle
)
```

Financial News Sentiment Analysis

❖ Step 6: Implement pytorch datasets and dataloader

```
class FinancialNews(Dataset):
    def __init__(
        self,
        X, y,
        word_to_idx,
        max_seq_len,
        transform=None
    ):
        self.texts = X
        self.labels = y
        self.word_to_idx = word_to_idx
        self.max_seq_len = max_seq_len
        self.transform = transform

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        label = self.labels[idx]

        if self.transform:
            text = self.transform(
                text,
                self.word_to_idx,
                self.max_seq_len
            )
        text = torch.tensor(text)

        return text, label
```

```
max_seq_len = 32

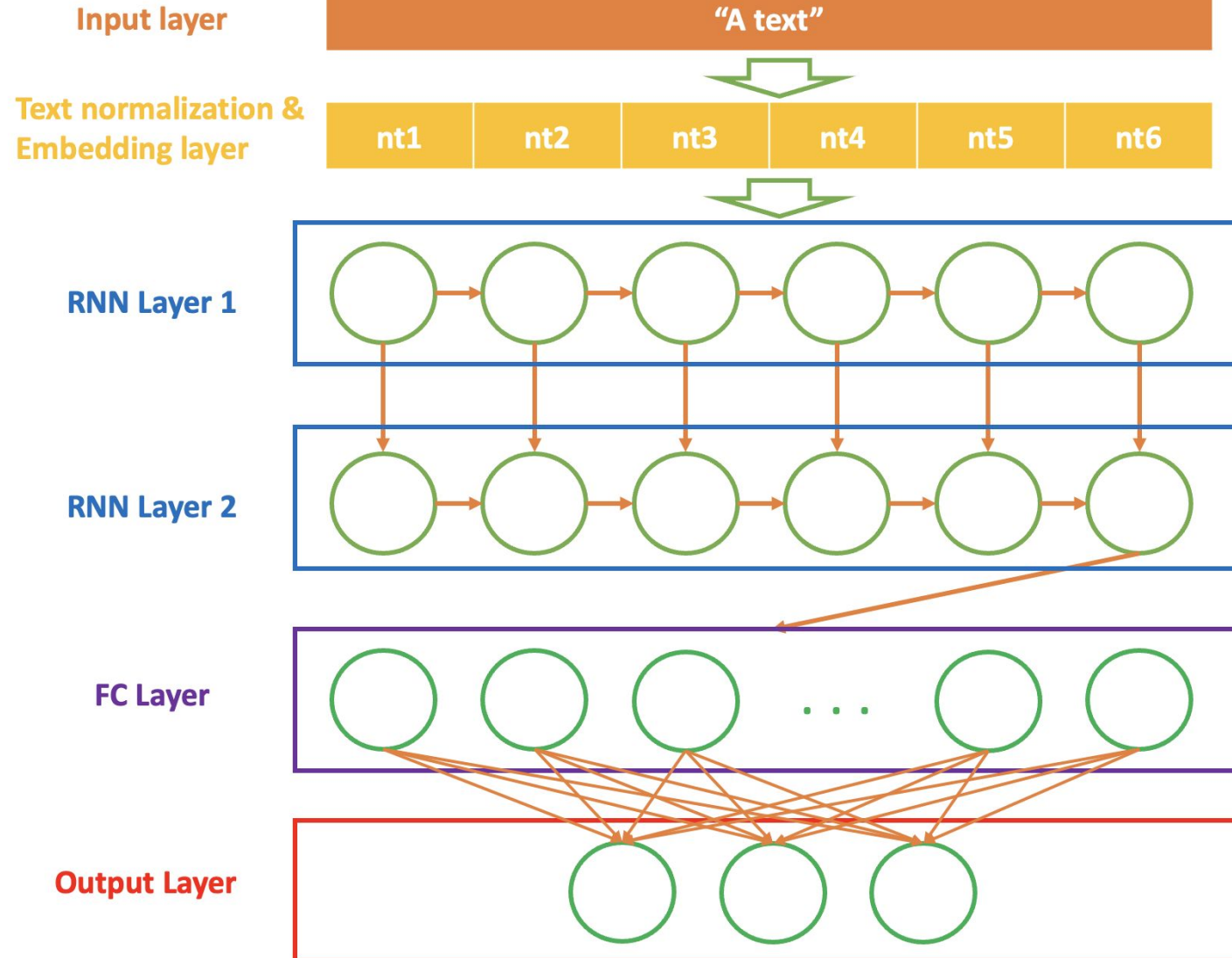
train_dataset = FinancialNews(
    X_train, y_train,
    word_to_idx=word_to_idx,
    max_seq_len=max_seq_len,
    transform=transform
)
val_dataset = FinancialNews(
    X_val, y_val,
    word_to_idx=word_to_idx,
    max_seq_len=max_seq_len,
    transform=transform
)
test_dataset = FinancialNews(
    X_test, y_test,
    word_to_idx=word_to_idx,
    max_seq_len=max_seq_len,
    transform=transform
)
```

```
train_batch_size = 128
test_batch_size = 8

train_loader = DataLoader(
    train_dataset,
    batch_size=train_batch_size,
    shuffle=True
)
val_loader = DataLoader(
    val_dataset,
    batch_size=test_batch_size,
    shuffle=False
)
test_loader = DataLoader(
    test_dataset,
    batch_size=test_batch_size,
    shuffle=False
)
```

Financial News Sentiment Analysis

◆ Step 7: Implement model



Financial News Sentiment Analysis

❖ Step 7: Implement model

Input text = "hello world"

1

5



Padding

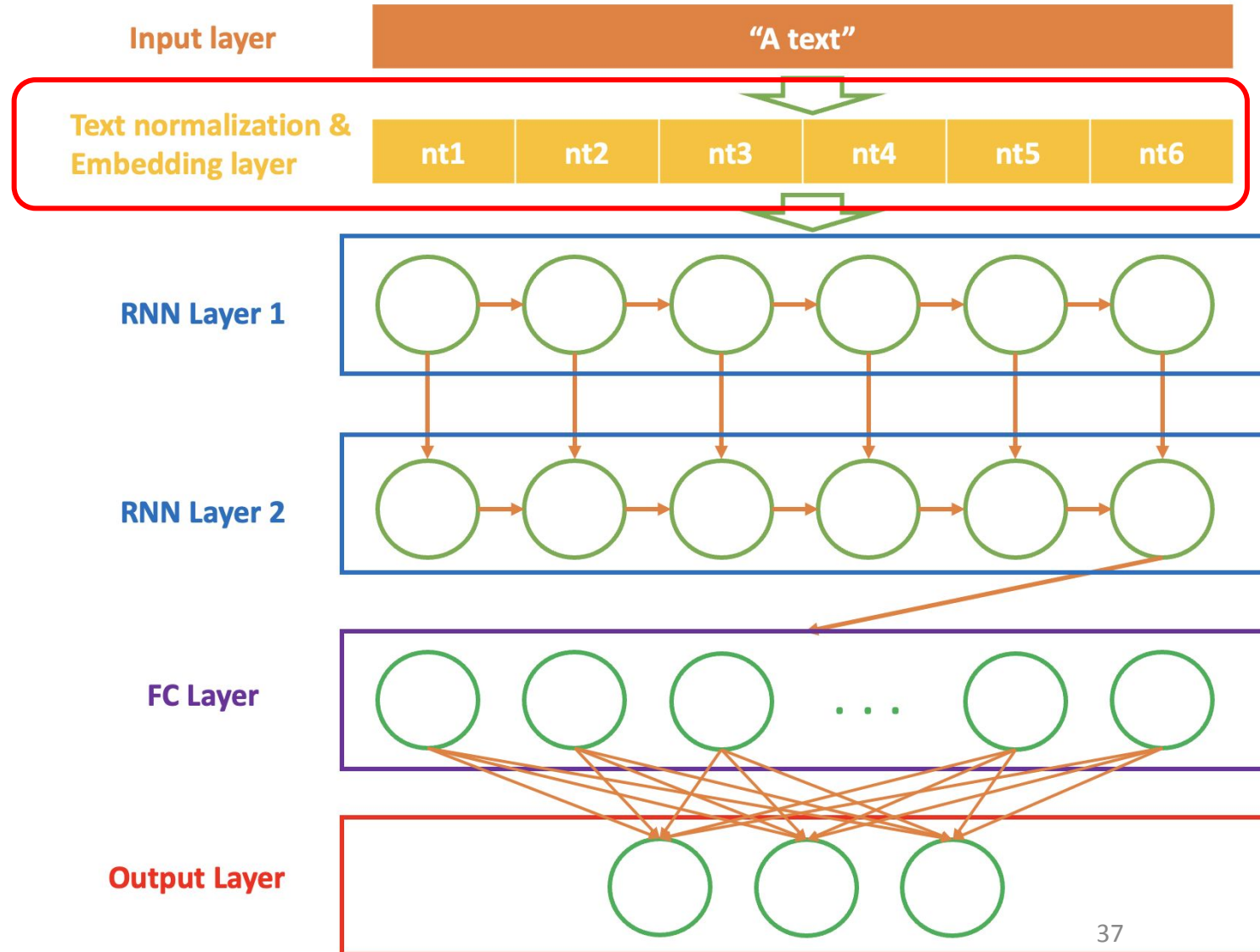
1

5

10

10

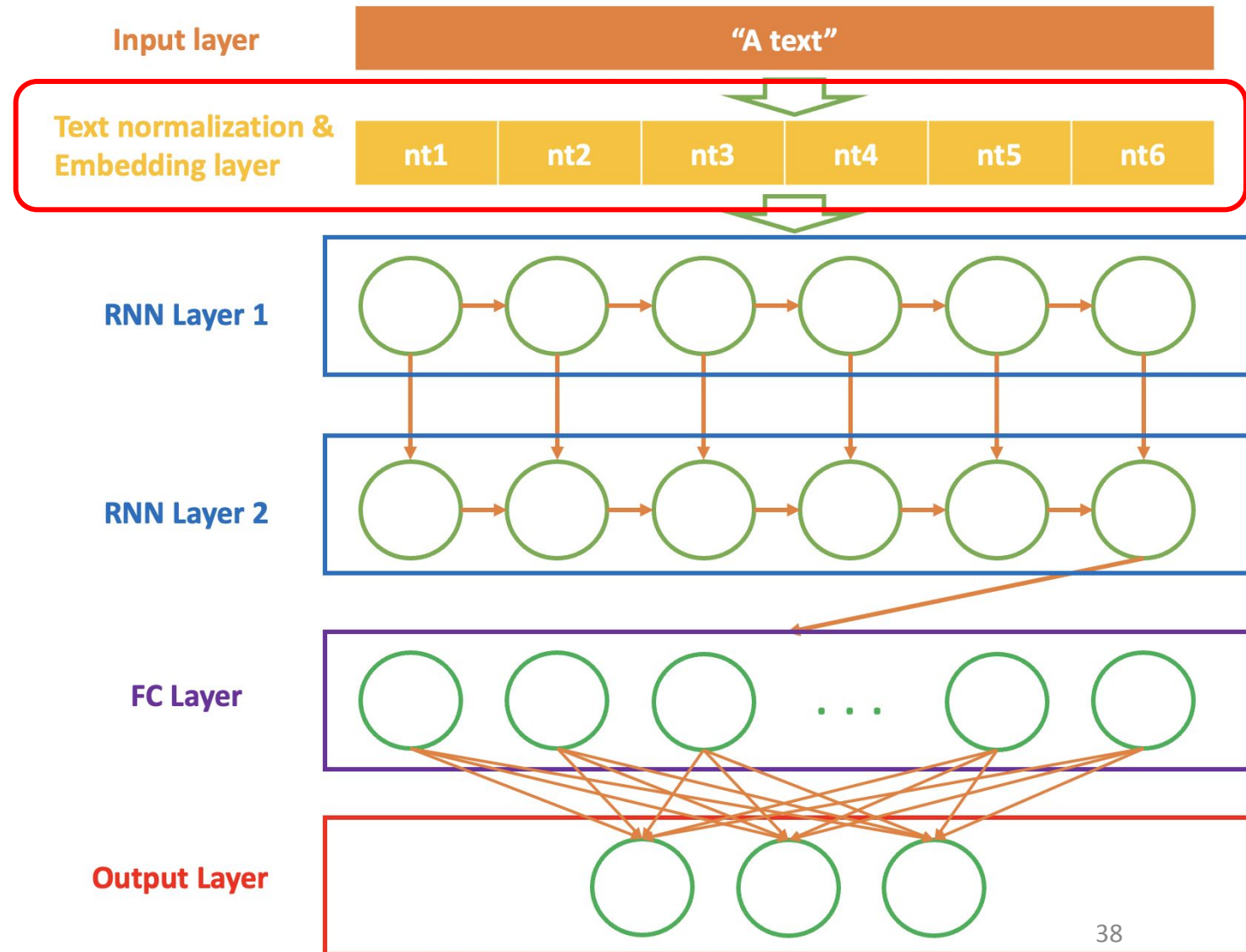
10



Financial News Sentiment Analysis

❖ Step 7: Implement model

1	5
0.04502351	-0.04007018
0.00151128	0.02874336
0.01764284	0.02772436
-0.0089057	0.00842067



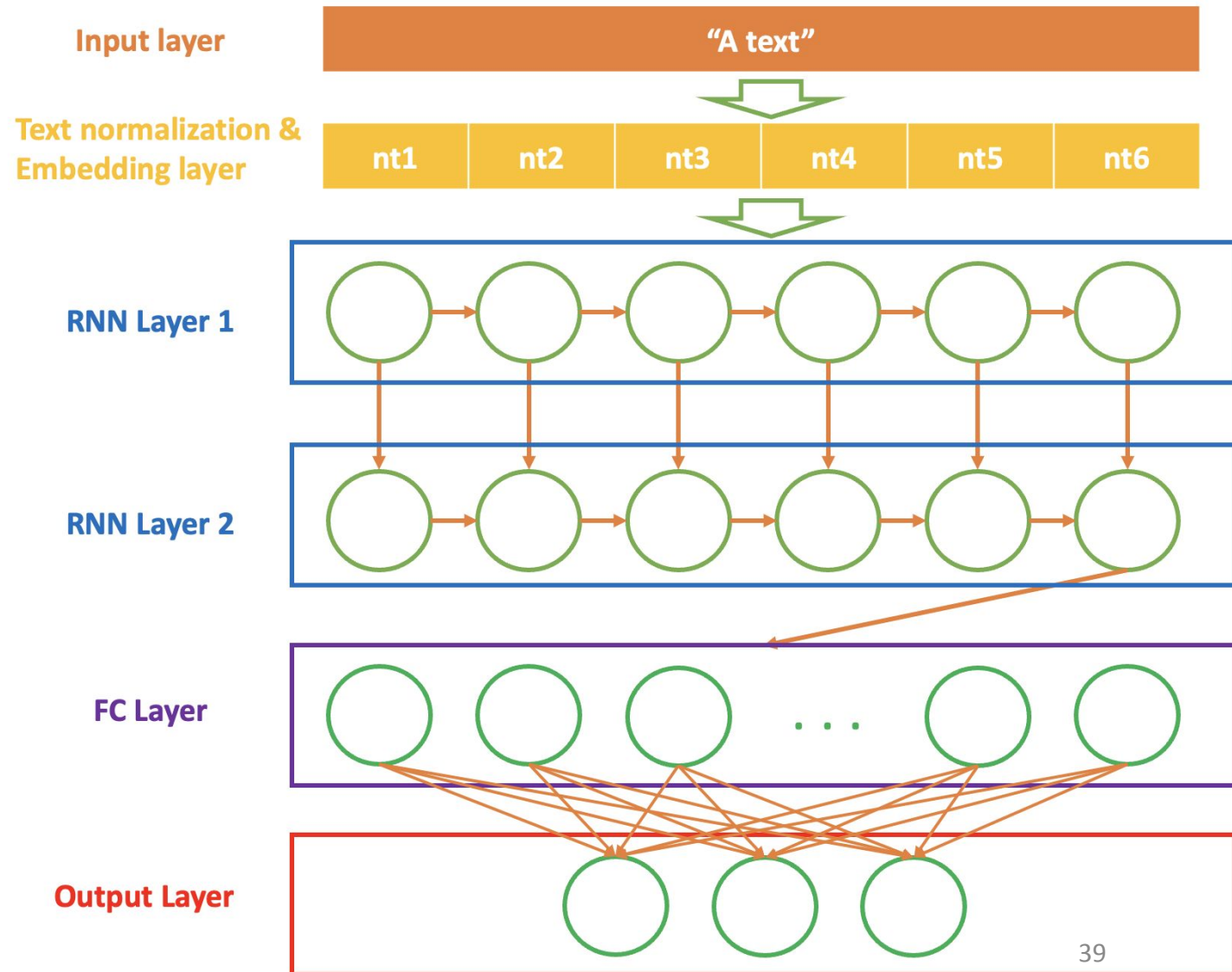
Financial News Sentiment Analysis

❖ Step 7: Implement model

```
class SentimentClassifier(nn.Module):
    def __init__(
        self, vocab_size, embedding_dim,
        hidden_size, n_layers, n_classes,
        dropout_prob
    ):
        super(SentimentClassifier, self).__init__()
        self.embedding = nn.Embedding(
            vocab_size, embedding_dim
        )
        self.rnn = nn.RNN(
            embedding_dim,
            hidden_size,
            n_layers,
            batch_first=True
        )
        self.norm = nn.LayerNorm(hidden_size)
        self.dropout = nn.Dropout(dropout_prob)
        self.fc1 = nn.Linear(hidden_size, 16)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(16, n_classes)

    def forward(self, x):
        x = self.embedding(x)
        x, hn = self.rnn(x)
        x = x[:, -1, :]
        x = self.norm(x)
        x = self.dropout(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)

        return x
```

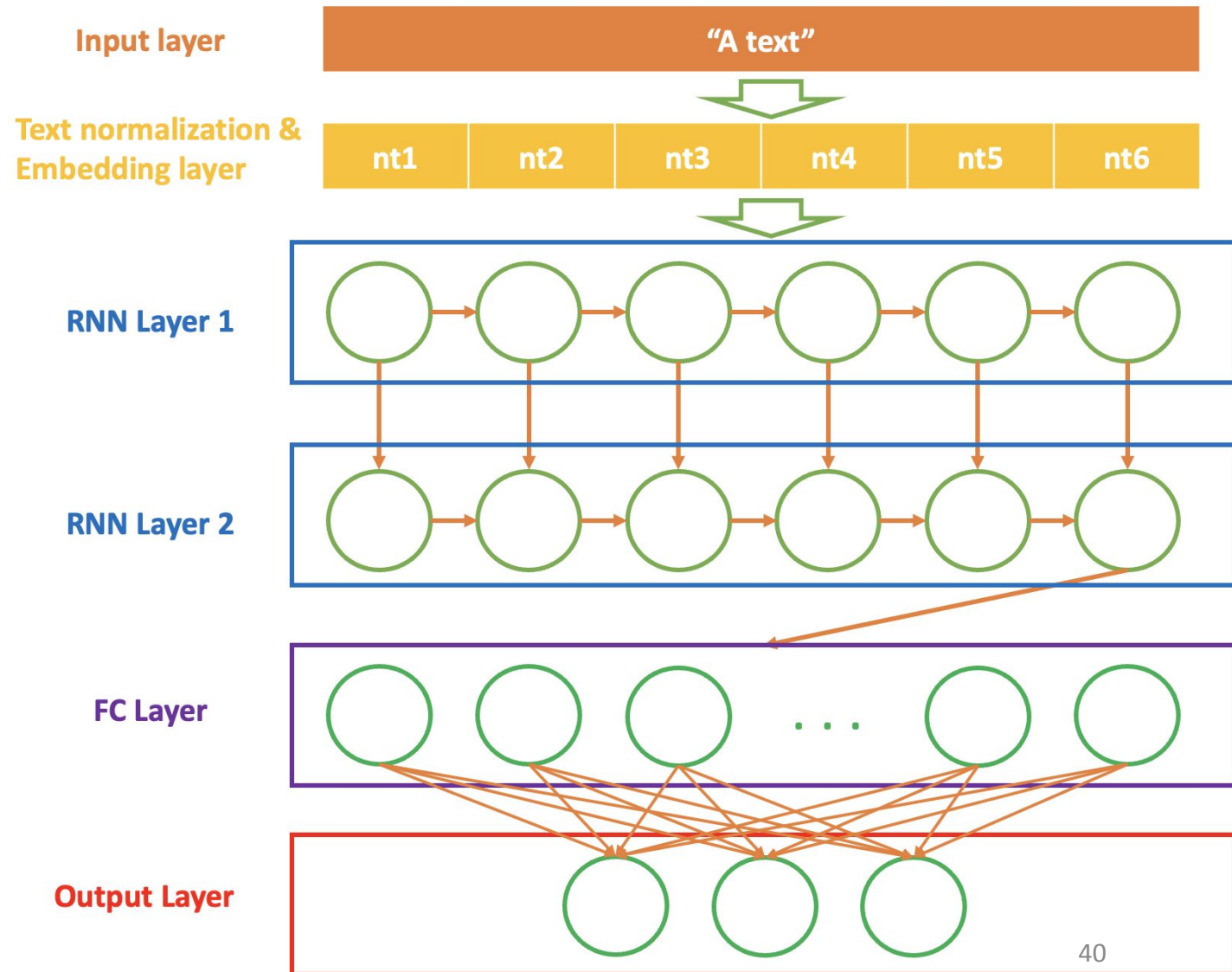


Financial News Sentiment Analysis

❖ Step 7: Implement model

```
n_classes = len(list(classes.keys()))
embedding_dim = 64
hidden_size = 64
n_layers = 2
dropout_prob = 0.2
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = SentimentClassifier(
    vocab_size=vocab_size,
    embedding_dim=embedding_dim,
    hidden_size=hidden_size,
    n_layers=n_layers,
    n_classes=n_classes,
    dropout_prob=dropout_prob
).to(device)
```



Financial News Sentiment Analysis

❖ Step 8: Create fit function

From line 1 to line 9: Declare fit() function with the following parameters:

- model: the pytorch model to use.
- train_loader: the dataloader of train set.
- val_loader: the dataloader of val set.
- criterion: the pytorch loss function.
- optimizer: the pytorch learning algorithm.
- device: 'cuda' or 'cpu'.
- epochs: number of epochs.

```
1 def fit(  
2     model,  
3     train_loader,  
4     val_loader,  
5     criterion,  
6     optimizer,  
7     device,  
8     epochs  
9 ):  
10     train_losses = []  
11     val_losses = []  
12  
13     for epoch in range(epochs):  
14         batch_train_losses = []  
15  
16         model.train()  
17         for idx, (inputs, labels) in enumerate(train_loader):  
18             inputs, labels = inputs.to(device), labels.to(device)  
19  
20             optimizer.zero_grad()  
21             outputs = model(inputs)  
22             loss = criterion(outputs, labels)  
23             loss.backward()  
24             optimizer.step()  
25  
26             batch_train_losses.append(loss.item())  
27  
28         train_loss = sum(batch_train_losses) / len(batch_train_losses)  
29         train_losses.append(train_loss)  
30  
31         val_loss, val_acc = evaluate(  
32             model, val_loader,  
33             criterion, device  
34         )  
35         val_losses.append(val_loss)  
36  
37         print(f'EPOCH {epoch + 1}: \tTrain loss: {train_loss:.4f} \tVal loss: {val_loss:.4f}')  
38  
39     return train_losses, val_losses
```

Financial News Sentiment Analysis

❖ Step 8: Create fit function

Line 10, 11: Declare two empty lists for storing losses on train set and val set over epochs. These will be the return values of our function.

```
1 def fit(  
2     model,  
3     train_loader,  
4     val_loader,  
5     criterion,  
6     optimizer,  
7     device,  
8     epochs  
9 ):  
10     train_losses = []  
11     val_losses = []  
12  
13     for epoch in range(epochs):  
14         batch_train_losses = []  
15  
16         model.train()  
17         for idx, (inputs, labels) in enumerate(train_loader):  
18             inputs, labels = inputs.to(device), labels.to(device)  
19  
20             optimizer.zero_grad()  
21             outputs = model(inputs)  
22             loss = criterion(outputs, labels)  
23             loss.backward()  
24             optimizer.step()  
25  
26             batch_train_losses.append(loss.item())  
27  
28         train_loss = sum(batch_train_losses) / len(batch_train_losses)  
29         train_losses.append(train_loss)  
30  
31         val_loss, val_acc = evaluate(  
32             model, val_loader,  
33             criterion, device  
34         )  
35         val_losses.append(val_loss)  
36  
37         print(f'EPOCH {epoch + 1}: \tTrain loss: {train_loss:.4f} \tVal loss: {val_loss:.4f}')  
38  
39     return train_losses, val_losses
```

Financial News Sentiment Analysis

❖ Step 8: Create fit function

Line 13, 14, 15, 16: Iterate over each epoch, create an empty list for storing loss on each iteration and turn the training mode of the model on.

```
1 def fit(  
2     model,  
3     train_loader,  
4     val_loader,  
5     criterion,  
6     optimizer,  
7     device,  
8     epochs  
9 ):  
10     train_losses = []  
11     val_losses = []  
12  
13     for epoch in range(epochs):  
14         batch_train_losses = []  
15  
16         model.train()  
17  
18         for idx, (inputs, labels) in enumerate(train_loader):  
19             inputs, labels = inputs.to(device), labels.to(device)  
20  
21             optimizer.zero_grad()  
22             outputs = model(inputs)  
23             loss = criterion(outputs, labels)  
24             loss.backward()  
25             optimizer.step()  
26  
27             batch_train_losses.append(loss.item())  
28  
29             train_loss = sum(batch_train_losses) / len(batch_train_losses)  
30             train_losses.append(train_loss)  
31  
32             val_loss, val_acc = evaluate(  
33                 model, val_loader,  
34                 criterion, device  
35             )  
36             val_losses.append(val_loss)  
37  
38             print(f'EPOCH {epoch + 1}: \tTrain loss: {train_loss:.4f} \tVal loss: {val_loss:.4f}')  
39  
40     return train_losses, val_losses
```

Financial News Sentiment Analysis

❖ Step 8: Create fit function

Line 17, 18: Iterate over each batch of samples in train dataloader, convert the input tensor into corresponding device (CPU or GPU).

Line 20, 21: Clear the previous gradient and execute the prediction on inputs.

```
1 def fit(  
2     model,  
3     train_loader,  
4     val_loader,  
5     criterion,  
6     optimizer,  
7     device,  
8     epochs  
9 ):  
10     train_losses = []  
11     val_losses = []  
12  
13     for epoch in range(epochs):  
14         batch_train_losses = []  
15  
16         model.train()  
17         for idx, (inputs, labels) in enumerate(train_loader):  
18             inputs, labels = inputs.to(device), labels.to(device)  
19  
20             optimizer.zero_grad()  
21             outputs = model(inputs)  
22             loss = criterion(outputs, labels)  
23             loss.backward()  
24             optimizer.step()  
25  
26             batch_train_losses.append(loss.item())  
27  
28         train_loss = sum(batch_train_losses) / len(batch_train_losses)  
29         train_losses.append(train_loss)  
30  
31         val_loss, val_acc = evaluate(  
32             model, val_loader,  
33             criterion, device  
34         )  
35         val_losses.append(val_loss)  
36  
37         print(f'EPOCH {epoch + 1}: \tTrain loss: {train_loss:.4f} \tVal loss: {val_loss:.4f}')  
38  
39     return train_losses, val_losses
```


Financial News Sentiment Analysis

❖ Step 8: Create fit function

Line 22, 23, 24, 26: Compute the loss, gradient and update the computed gradient to the optimizer. Then, append the computed loss to empty list declared on line 14.

```
1 def fit(  
2     model,  
3     train_loader,  
4     val_loader,  
5     criterion,  
6     optimizer,  
7     device,  
8     epochs  
9 ):  
10     train_losses = []  
11     val_losses = []  
12  
13     for epoch in range(epochs):  
14         batch_train_losses = []  
15  
16         model.train()  
17         for idx, (inputs, labels) in enumerate(train_loader):  
18             inputs, labels = inputs.to(device), labels.to(device)  
19  
20             optimizer.zero_grad()  
21             outputs = model(inputs)  
22             loss = criterion(outputs, labels)  
23             loss.backward()  
24             optimizer.step()  
25  
26             batch_train_losses.append(loss.item())  
27  
28         train_loss = sum(batch_train_losses) / len(batch_train_losses)  
29         train_losses.append(train_loss)  
30  
31         val_loss, val_acc = evaluate(  
32             model, val_loader,  
33             criterion, device  
34         )  
35         val_losses.append(val_loss)  
36  
37         print(f'EPOCH {epoch + 1}: \tTrain loss: {train_loss:.4f} \tVal loss: {val_loss:.4f}')  
38  
39     return train_losses, val_losses
```

Financial News Sentiment Analysis

❖ Step 8: Create fit function

Line 28, 29: With a list of loss value of each iteration, we average them to get a single representative value for the current epoch.

```
1 def fit(  
2     model,  
3     train_loader,  
4     val_loader,  
5     criterion,  
6     optimizer,  
7     device,  
8     epochs  
9 ):  
10     train_losses = []  
11     val_losses = []  
12  
13     for epoch in range(epochs):  
14         batch_train_losses = []  
15  
16         model.train()  
17         for idx, (inputs, labels) in enumerate(train_loader):  
18             inputs, labels = inputs.to(device), labels.to(device)  
19  
20             optimizer.zero_grad()  
21             outputs = model(inputs)  
22             loss = criterion(outputs, labels)  
23             loss.backward()  
24             optimizer.step()  
25  
26             batch_train_losses.append(loss.item())  
27  
28             train_loss = sum(batch_train_losses) / len(batch_train_losses)  
29             train_losses.append(train_loss)  
30  
31         val_loss, val_acc = evaluate(  
32             model, val_loader,  
33             criterion, device  
34         )  
35         val_losses.append(val_loss)  
36  
37         print(f'EPOCH {epoch + 1}: \tTrain loss: {train_loss:.4f} \tVal loss: {val_loss:.4f}')  
38  
39     return train_losses, val_losses
```

Financial News Sentiment Analysis

❖ Step 8: Create fit function

From line 31 to 39: Evaluate the model on val set, append the val loss to empty list declared in line 11, print the training result on the current epoch. If the iteration over epoch ends, return the train_losses and val_losses.

```
1 def fit(  
2     model,  
3     train_loader,  
4     val_loader,  
5     criterion,  
6     optimizer,  
7     device,  
8     epochs  
9 ):  
10     train_losses = []  
11     val_losses = []  
12  
13     for epoch in range(epochs):  
14         batch_train_losses = []  
15  
16         model.train()  
17         for idx, (inputs, labels) in enumerate(train_loader):  
18             inputs, labels = inputs.to(device), labels.to(device)  
19  
20             optimizer.zero_grad()  
21             outputs = model(inputs)  
22             loss = criterion(outputs, labels)  
23             loss.backward()  
24             optimizer.step()  
25  
26             batch_train_losses.append(loss.item())  
27  
28         train_loss = sum(batch_train_losses) / len(batch_train_losses)  
29         train_losses.append(train_loss)  
30  
31         val_loss, val_acc = evaluate(  
32             model, val_loader,  
33             criterion, device  
34         )  
35         val_losses.append(val_loss)  
36  
37         print(f'EPOCH {epoch + 1}: \tTrain loss: {train_loss:.4f} \tVal loss: {val_loss:.4f}')  
38  
39     return train_losses, val_losses
```

Financial News Sentiment Analysis

❖ Step 9: Training

```
lr = 1e-4
epochs = 50

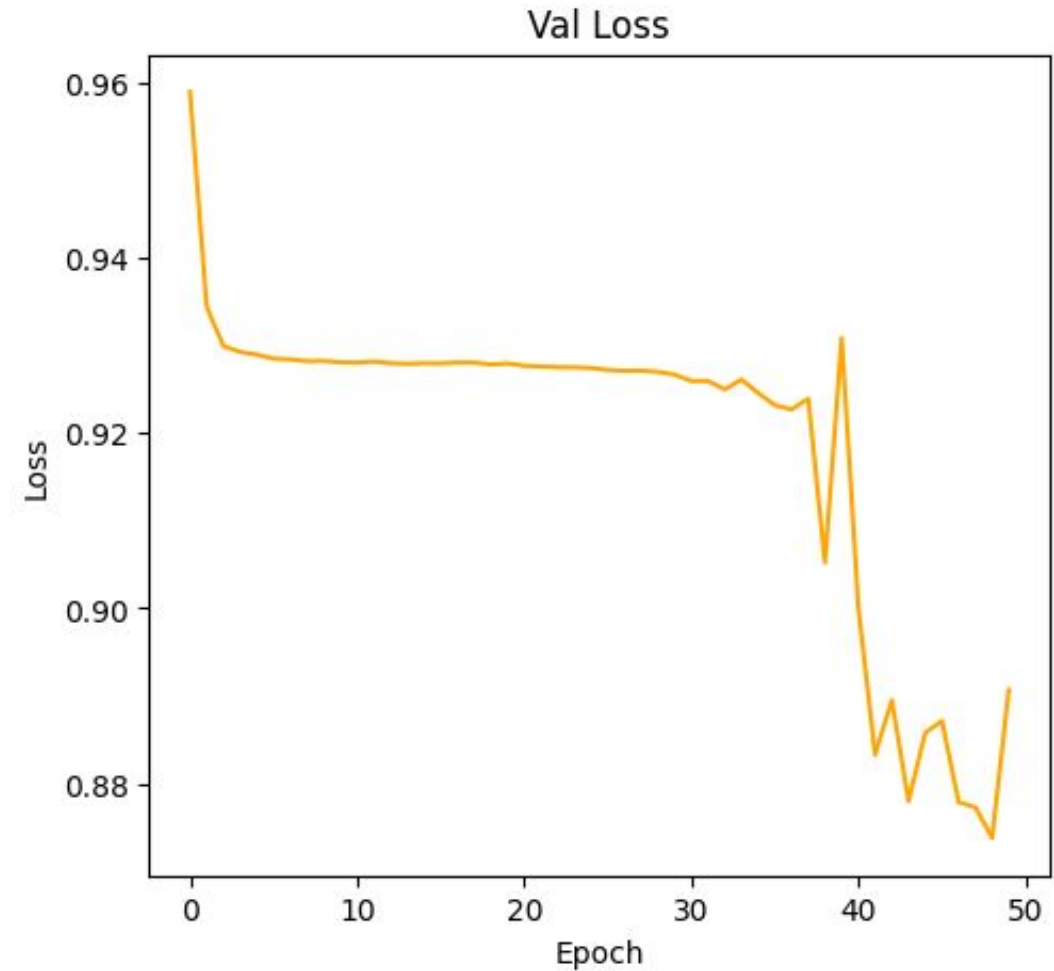
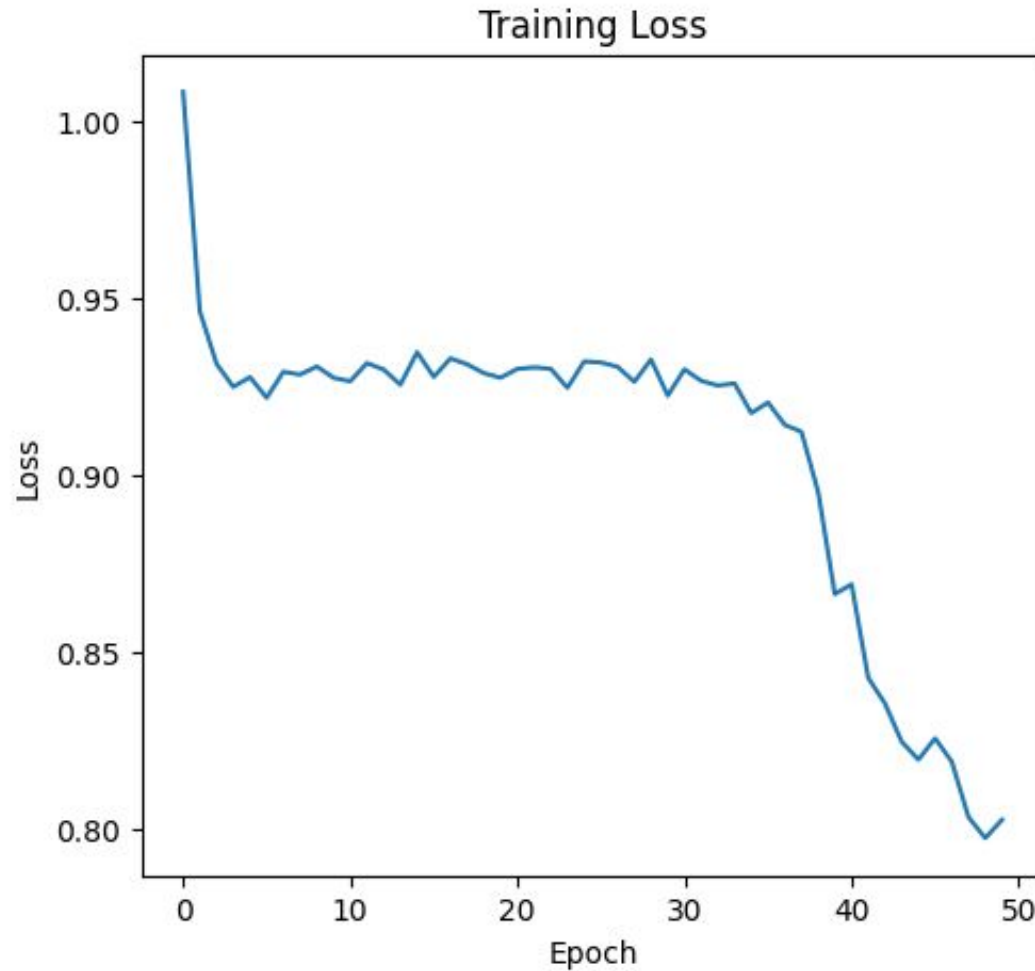
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=lr
)

train_losses, val_losses = fit(
    model,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    device,
    epochs
)
```

EPOCH 1:	Train loss: 1.9155	Val loss: 2.3871
EPOCH 2:	Train loss: 1.3232	Val loss: 2.3897
EPOCH 3:	Train loss: 1.0485	Val loss: 2.4028
EPOCH 4:	Train loss: 0.8511	Val loss: 2.0351
EPOCH 5:	Train loss: 0.6936	Val loss: 1.8467
EPOCH 6:	Train loss: 0.5541	Val loss: 1.5207
EPOCH 7:	Train loss: 0.4349	Val loss: 1.5914
EPOCH 8:	Train loss: 0.3392	Val loss: 1.2092
EPOCH 9:	Train loss: 0.2574	Val loss: 1.1194
EPOCH 10:	Train loss: 0.1973	Val loss: 1.1830
EPOCH 11:	Train loss: 0.1389	Val loss: 1.0327
EPOCH 12:	Train loss: 0.1029	Val loss: 1.0589
EPOCH 13:	Train loss: 0.0871	Val loss: 1.0458
EPOCH 14:	Train loss: 0.0667	Val loss: 1.0200
EPOCH 15:	Train loss: 0.0558	Val loss: 1.0302
EPOCH 16:	Train loss: 0.0464	Val loss: 1.0318
EPOCH 17:	Train loss: 0.0422	Val loss: 1.0393
EPOCH 18:	Train loss: 0.0348	Val loss: 1.0261
EPOCH 19:	Train loss: 0.0294	Val loss: 1.0330
EPOCH 20:	Train loss: 0.0274	Val loss: 1.0285
EPOCH 21:	Train loss: 0.0236	Val loss: 1.0346
EPOCH 22:	Train loss: 0.0206	Val loss: 1.0378
EPOCH 23:	Train loss: 0.0194	Val loss: 1.0406
EPOCH 24:	Train loss: 0.0178	Val loss: 1.0382
EPOCH 25:	Train loss: 0.0169	Val loss: 1.0332

Financial News Sentiment Analysis

◆ Step 9: Training



Financial News Sentiment Analysis

❖ Step 10: Create evaluation function

```
1 def evaluate(model, dataloader, criterion, device):
2     model.eval()
3     correct = 0
4     total = 0
5     losses = []
6     with torch.no_grad():
7         for inputs, labels in dataloader:
8             inputs, labels = inputs.to(device), labels.to(device)
9             outputs = model(inputs)
10            loss = criterion(outputs, labels)
11            losses.append(loss.item())
12            _, predicted = torch.max(outputs.data, 1)
13            total += labels.size(0)
14            correct += (predicted == labels).sum().item()
15
16    loss = sum(losses) / len(losses)
17    acc = correct / total
18
19    return loss, acc
```

```
val_loss, val_acc = evaluate(
    model,
    val_loader,
    criterion,
    device
)
test_loss, test_acc = evaluate(
    model,
    test_loader,
    criterion,
    device
)

print('Evaluation on val/test dataset')
print('Val accuracy: ', val_acc)
print('Test accuracy: ', test_acc)
```

```
Evaluation on val/test dataset
Val accuracy:  0.6144329896907217
Test accuracy: 0.6069587628865979
```

Financial News Sentiment Analysis

Input News

Enter your financial news text here:

With the new production plant the company would increase its capacity to meet the expected increase in demand and would improve the use of raw materials and therefore increase the production profitability .



Analyze Sentiment

Sentiment: Positive with 48.81% probability

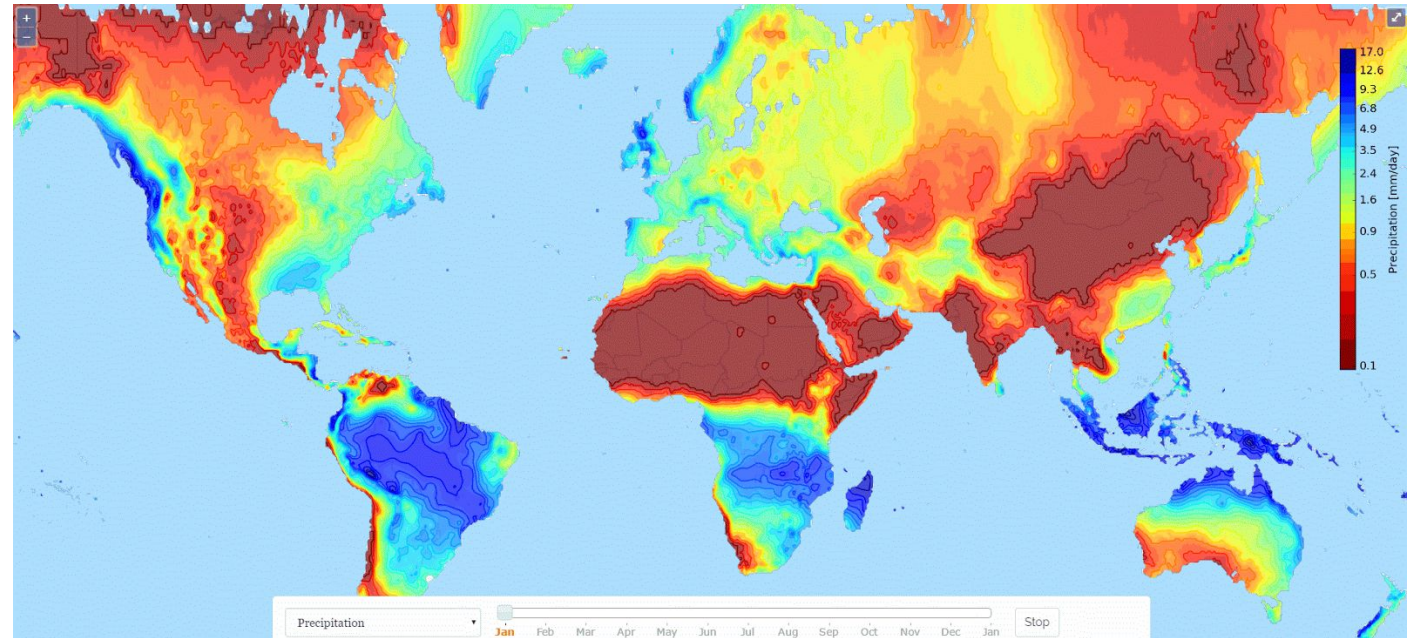
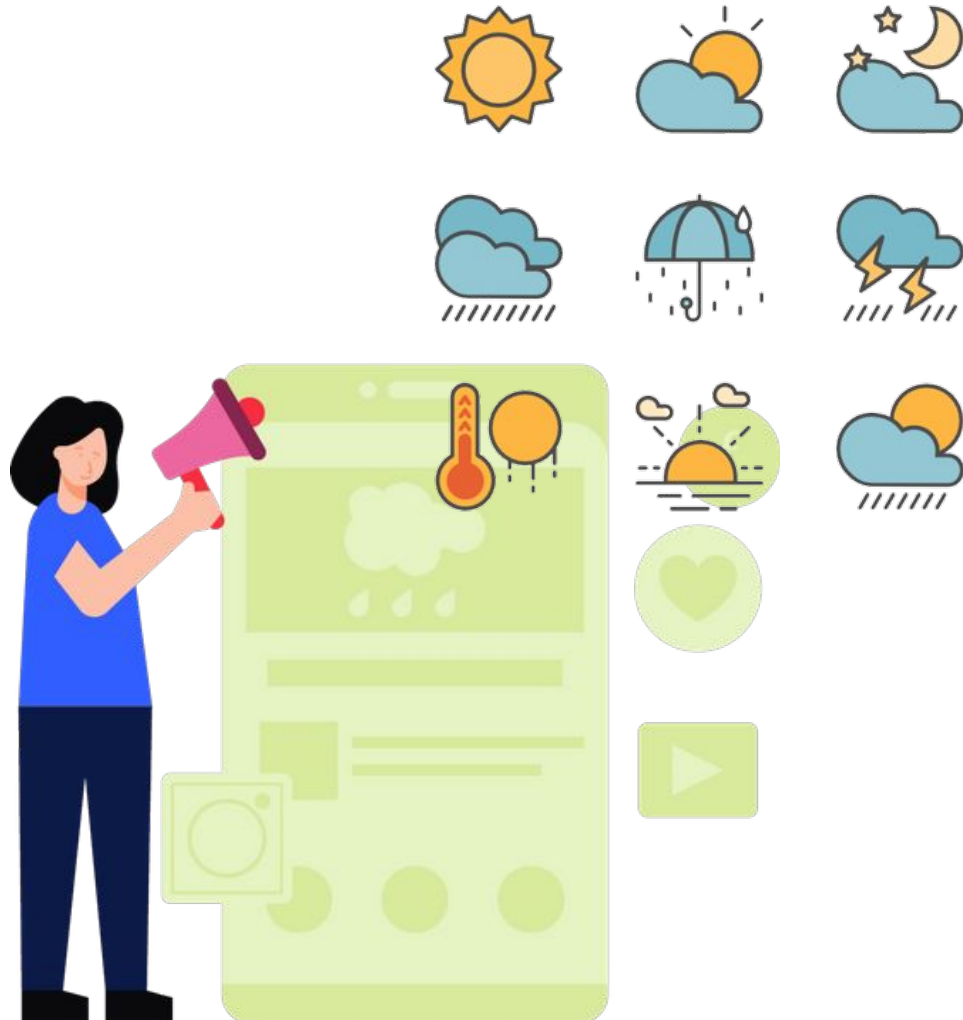
Quiz

Khi xây dựng một mô hình phân loại cảm xúc (sentiment classifier) sử dụng RNN, thành phần nào sau đây thường được sử dụng để biểu diễn từ vựng đầu vào?

- a) One-hot encoding**
- b) Embedding layer**
- c) Convolutional layer**
- d) Batch normalization**

Temperature Forecasting

❖ Coding Exercise



Predict future temperature in weather forecasting

Temperature Forecasting

❖ Coding Exercise

Problem Statement: Given a dataset recorded temperature per hour (download [here](#)) Build a model that given temperature from the **previous 6 hours** (including the current one), predict temperature of the **next 1 hour**.

Hour	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00
Condition								
Temperature	32	31	31	30	29	26	25	

Temperature Forecasting

❖ Step 1: Import libraries

```
import torch
import torch.nn as nn

seed = 1
torch.manual_seed(seed)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from torch.utils.data import Dataset, DataLoader
```



Temperature Forecasting

◆ Step 2: Read dataset

	Formatted Date	Summary	Precip Type	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	Wind Bearing (degrees)	Visibility (km)	Loud Cover	Pressure (millibars)	Daily Summary
0	2006-04-01 00:00:00.000 +0200	Partly Cloudy	rain	9.472222	7.388889	0.89	14.1197	251.0	15.8263	0.0	1015.13	Partly cloudy throughout the day.
1	2006-04-01 01:00:00.000 +0200	Partly Cloudy	rain	9.355556	7.227778	0.86	14.2646	259.0	15.8263	0.0	1015.63	Partly cloudy throughout the day.
2	2006-04-01 02:00:00.000 +0200	Mostly Cloudy	rain	9.377778	9.377778	0.89	3.9284	204.0	14.9569	0.0	1015.94	Partly cloudy throughout the day.
3	2006-04-01 03:00:00.000 +0200	Partly Cloudy	rain	8.288889	5.944444	0.83	14.1036	269.0	15.8263	0.0	1016.41	Partly cloudy throughout the day.
4	2006-04-01 04:00:00.000 +0200	Mostly Cloudy	rain	8.755556	6.977778	0.83	11.0446	259.0	15.8263	0.0	1016.51	Partly cloudy throughout the day.
...	...	...	...	...	...	...	...	...	...	...	...	...

- ◆ **Columns:** Date, Summary, Precip Type, Temperature, Apparent Temperature, Humidity, Wind Speed, Wind Bearing, Visibility, Loud Cover, Pressure, Daily Summary (12 columns).
- ◆ **Number of samples:** 96453.

Temperature Forecasting

❖ Step 2: Read dataset

	Formatted Date	Summary	Precip Type	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	Wind Bearing (degrees)	Visibility (km)	Loud Cover	Pressure (millibars)	Daily Summary
0	2006-04-01 00:00:00.000 +0200	Partly Cloudy	rain	9.472222	7.388889	0.89	14.1197	251.0	15.8263	0.0	1015.13	Partly cloudy throughout the day.
1	2006-04-01 01:00:00.000 +0200	Partly Cloudy	rain	9.355556	7.227778	0.86	14.2646					Partly cloudy throughout the day.
2	2006-04-01 02:00:00.000 +0200	Mostly Cloudy	rain	9.377778	9.377778	0.89	3.9284					Partly cloudy throughout the day.
3	2006-04-01 03:00:00.000 +0200	Partly Cloudy	rain	8.288889	5.944444	0.83	14.1036	269.0	15.8263	0.0	1016.41	Partly cloudy throughout the day.
4	2006-04-01 04:00:00.000 +0200	Mostly Cloudy	rain	8.755556	6.977778	0.83	11.0446	259.0	15.8263	0.0	1016.51	Partly cloudy throughout the day.
...	...	...	...	...	...	...	...	...	...	...	...	...

Consider only these two columns

❖ **New Columns:** Date, Temperature (2 columns).

❖ **Number of samples:** 96453.

Temperature Forecasting

◆ Step 2: Read dataset

```
Formatted Date
2006-04-01 00:00:00.000 +0200      9.472222
2006-04-01 01:00:00.000 +0200      9.355556
2006-04-01 02:00:00.000 +0200      9.377778
2006-04-01 03:00:00.000 +0200      8.288889
2006-04-01 04:00:00.000 +0200      8.755556
...
2016-09-09 19:00:00.000 +0200     26.016667
2016-09-09 20:00:00.000 +0200     24.583333
2016-09-09 21:00:00.000 +0200     22.038889
2016-09-09 22:00:00.000 +0200     21.522222
2016-09-09 23:00:00.000 +0200     20.438889
Name: Temperature (C), Length: 96453, dtype: float64
```

```
1 dataset_filepath = './weatherHistory.csv'
2 df = pd.read_csv(dataset_filepath)
3
4
5
6 univariate_df = df['Temperature (C)']
7 univariate_df.index = df['Formatted Date']
8 univariate_df
```


Temperature Forecasting

❖ Step 2: Read dataset

Formatted Date

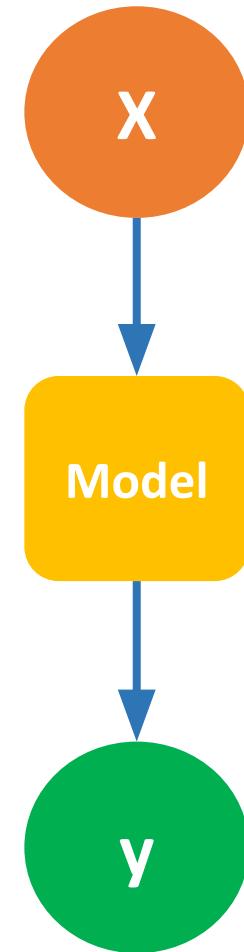
2006-04-01	00:00:00.000	+0200	9.472222
2006-04-01	01:00:00.000	+0200	9.355556
2006-04-01	02:00:00.000	+0200	9.377778
2006-04-01	03:00:00.000	+0200	8.288889
2006-04-01	04:00:00.000	+0200	8.755556

...

2016-09-09	19:00:00.000	+0200	26.016667
2016-09-09	20:00:00.000	+0200	24.583333
2016-09-09	21:00:00.000	+0200	22.038889
2016-09-09	22:00:00.000	+0200	21.522222
2016-09-09	23:00:00.000	+0200	20.438889

Name: Temperature (C), Length: 96453, dtype: float64

Not in proper format yet



Temperature Forecasting

❖ Step 3: Prepare (X, y) samples

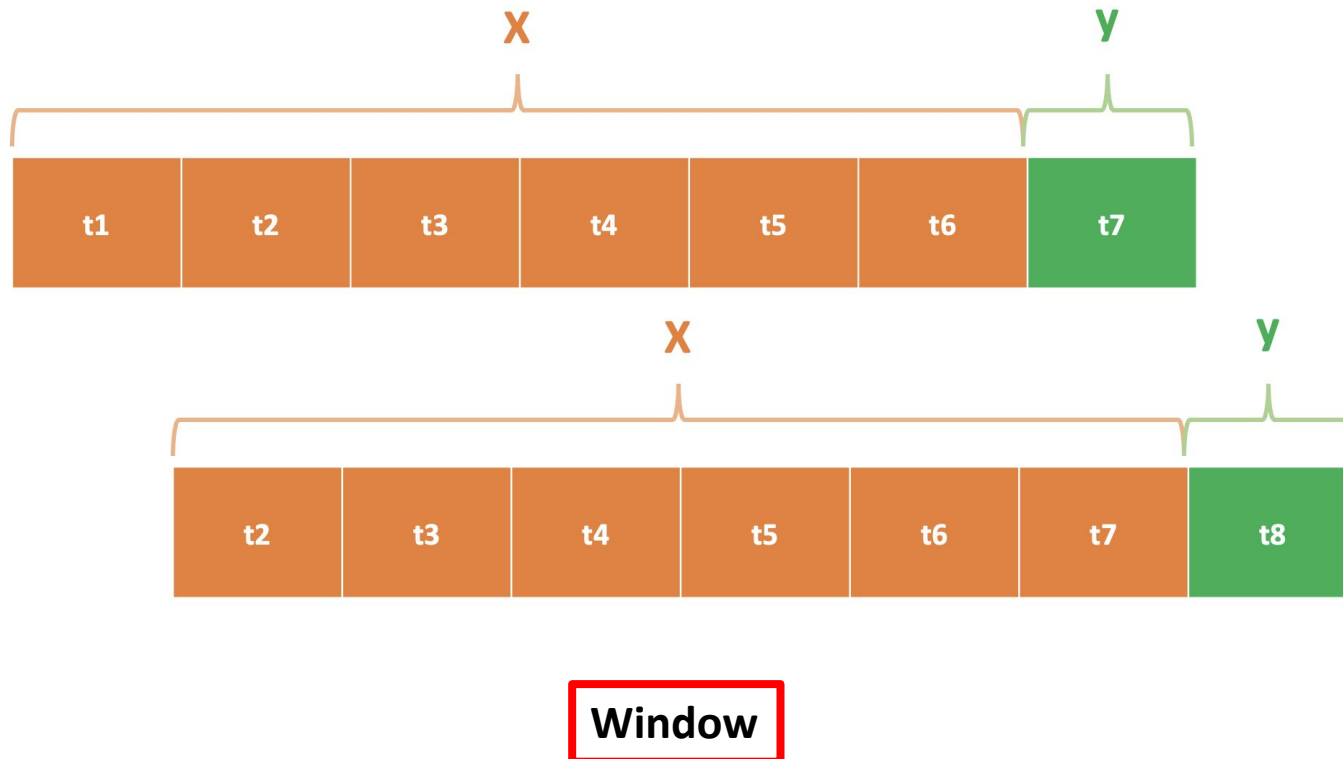
Problem Statement: Given temperature from the **previous 6 hours** (including the current one), predict temperature of the **next 1 hour**.

Time	Temperature (C)
2006-04-01 00:00:00.000 +0200	9.472222
2006-04-01 01:00:00.000 +0200	9.355556
2006-04-01 02:00:00.000 +0200	9.377778
2006-04-01 03:00:00.000 +0200	8.288889
2006-04-01 04:00:00.000 +0200	8.755556
2006-04-01 05:00:00.000 +0200	9.222222
2006-04-01 06:00:00.000 +0200	7.733333



Temperature Forecasting

❖ Step 3: Prepare (X, y) samples: Sliding Window



Sliding window technique:

- ❖ Iterate through each row in datatable.
- ❖ Select 7 rows.
- ❖ Extract:
 - X: temperature of the first 6 rows.
 - y: temperature of the last row.

Temperature Forecasting

◆ Step 3: Prepare (X, y) samples

```
1 def slicing_window(df, df_start_idx, df_end_idx, input_size, label_size, offset):
2     features = []
3     labels = []
4
5     window_size = input_size + offset
6
7     if df_end_idx == None:
8         df_end_idx = len(df) - window_size
9
10    for idx in range(df_start_idx, df_end_idx):
11        feature_end_idx = idx + input_size
12        label_start_idx = idx + window_size - label_size
13
14        feature = df[idx:feature_end_idx]
15        label = df[label_start_idx:(idx+window_size)]
16
17        features.append(feature)
18        labels.append(label)
19
20    features = np.expand_dims(np.array(features), -1)
21    labels = np.array(labels)
22
23    return features, labels
```

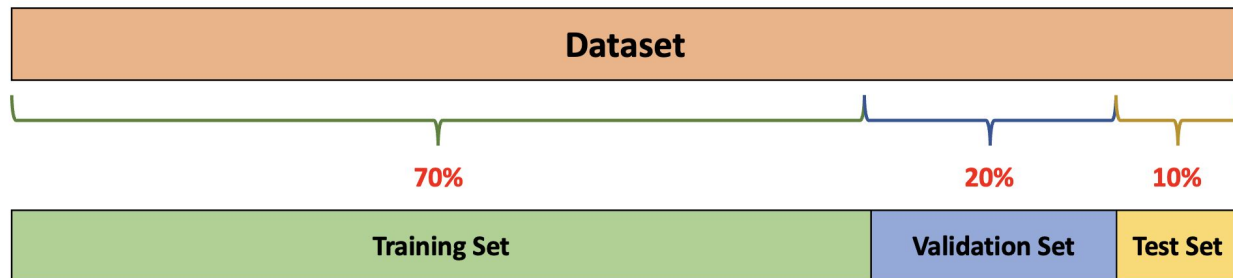
```
1 INPUT_SIZE = 6
2 LABEL_SIZE = 1
3 OFFSET = 1
4 START_IDX = 0
5 END_IDX = 1
6
7 X, y = slicing_window(univariate_df,
8                       df_start_idx=START_IDX,
9                       df_end_idx=END_IDX,
10                      input_size=INPUT_SIZE,
11                      label_size=LABEL_SIZE,
12                      offset=OFFSET)
```

```
X: [[[9.47222222]
      [9.35555556]
      [9.37777778]
      [8.28888889]
      [8.75555556]
      [9.22222222]]], shape: (1, 6, 1)
y: [[7.73333333]], shape: (1, 1)
```

Temperature Forecasting

❖ Step 4: Create train, val, test set

```
dataset_length = len(univariate_df)
train_size = 0.7
val_size = 0.2
train_end_idx = int(train_size * dataset_length)
val_end_idx = int(val_size * dataset_length) + train_end_idx
```



```
X_train, y_train = slicing_window(
    univariate_df,
    df_start_idx=0,
    df_end_idx=train_end_idx,
    input_size=input_size,
    label_size=label_size,
    offset=offset
)
```

```
X_val, y_val = slicing_window(
    univariate_df,
    df_start_idx=train_end_idx,
    df_end_idx=val_end_idx,
    input_size=input_size,
    label_size=label_size,
    offset=offset
)
```

```
X_test, y_test = slicing_window(
    univariate_df,
    df_start_idx=val_end_idx,
    df_end_idx=None,
    input_size=input_size,
    label_size=label_size,
    offset=offset
)
```


Temperature Forecasting

❖ Step 5: Create pytorch datasets and dataloader

```
class WeatherForecast(Dataset):
    def __init__(
        self,
        X, y,
        transform=None
    ):
        self.X = X
        self.y = y
        self.transform = transform

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        X = self.X[idx]
        y = self.y[idx]

        if self.transform:
            X = self.transform(X)

        X = torch.tensor(X, dtype=torch.float32)
        y = torch.tensor(y, dtype=torch.float32)

        return X, y
```

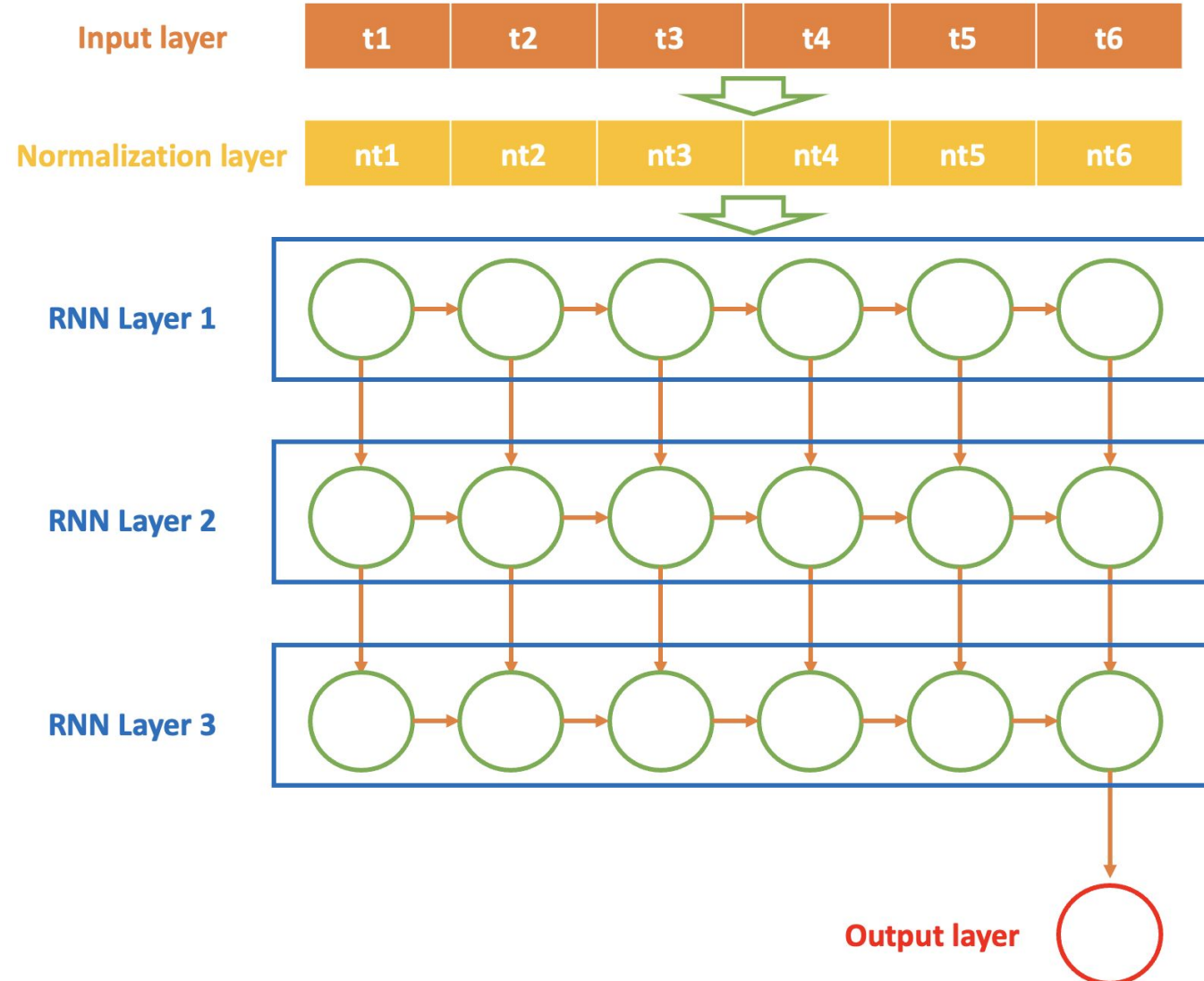
```
train_dataset = WeatherForecast(
    X_train, y_train
)
val_dataset = WeatherForecast(
    X_val, y_val
)
test_dataset = WeatherForecast(
    X_test, y_test
)
```

```
train_batch_size = 128
test_batch_size = 8

train_loader = DataLoader(
    train_dataset,
    batch_size=train_batch_size,
    shuffle=True
)
val_loader = DataLoader(
    val_dataset,
    batch_size=test_batch_size,
    shuffle=False
)
test_loader = DataLoader(
    test_dataset,
    batch_size=test_batch_size,
    shuffle=False
)
```

Temperature Forecasting

❖ Step 6: Implement model

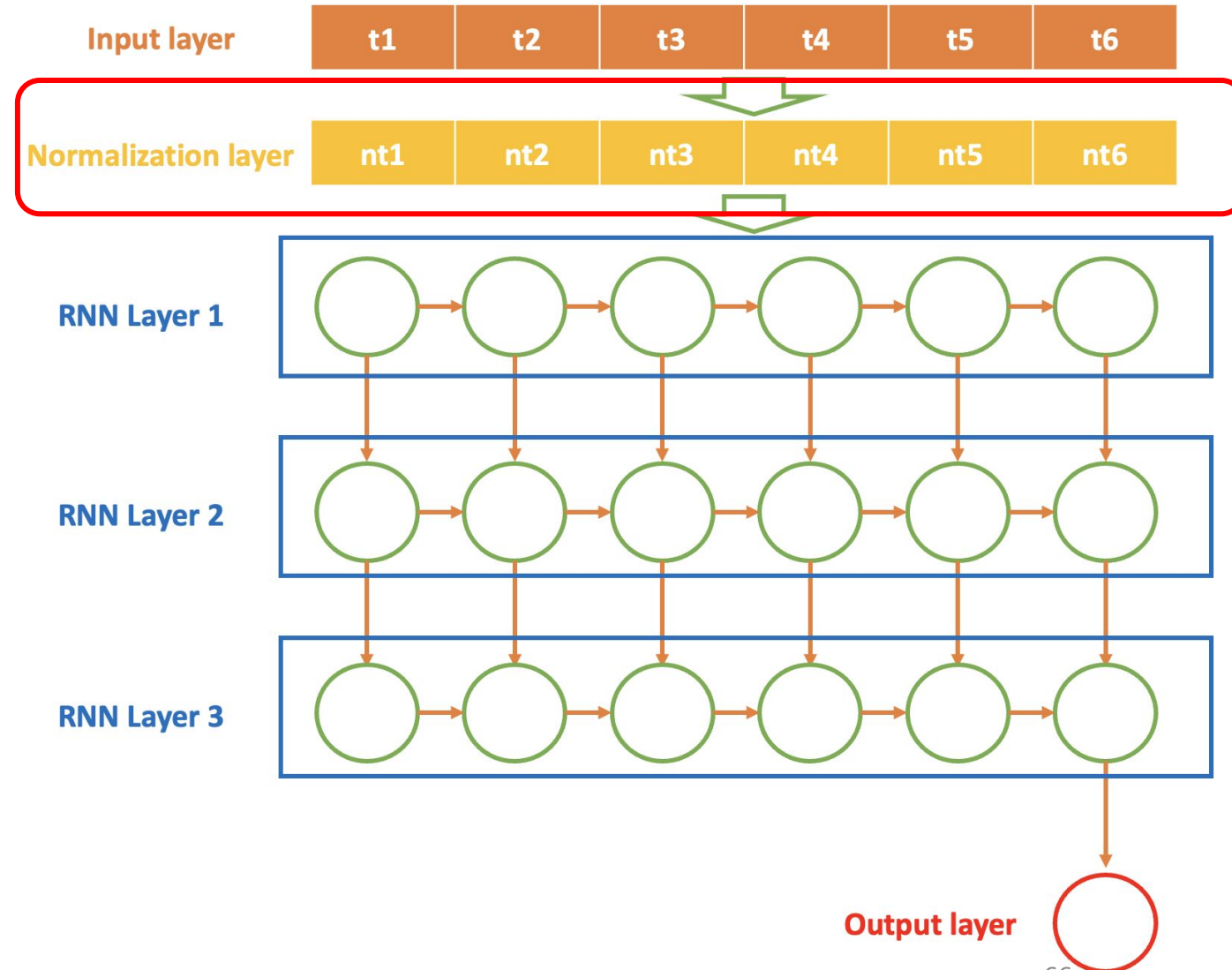


Temperature Forecasting

❖ Step 6: Implement model

$$X_{norm} = \frac{X - \mu}{\sigma} \quad \begin{matrix} \diamond \mu: \text{mean of } X \\ \diamond \sigma: \text{standard deviation of } X \end{matrix}$$

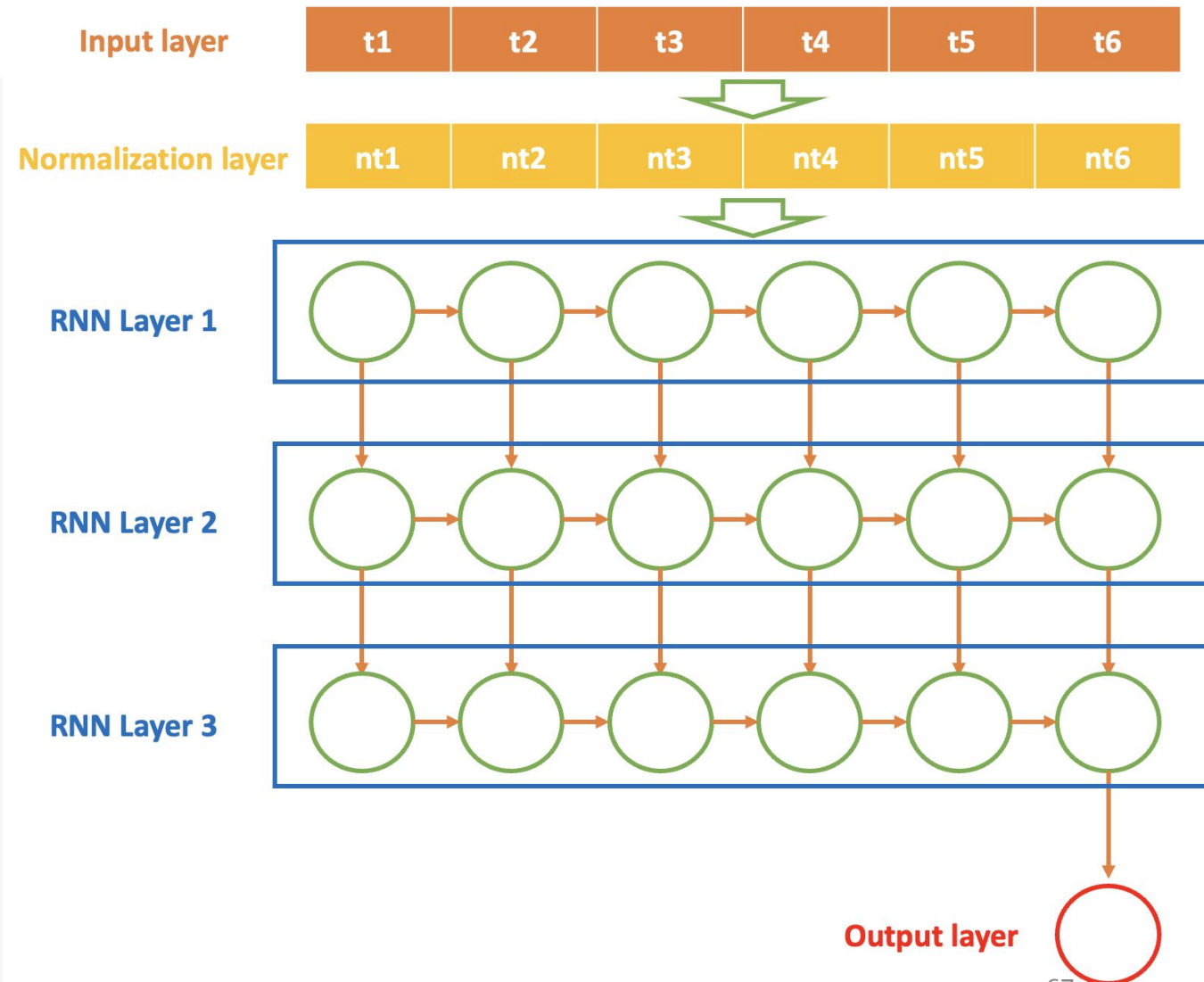
```
Formatted Date
2006-04-01 00:00:00.000 +0200    9.472222
2006-04-01 01:00:00.000 +0200    9.355556
2006-04-01 02:00:00.000 +0200    9.377778
2006-04-01 03:00:00.000 +0200    8.288889
2006-04-01 04:00:00.000 +0200    8.755556
...
2016-09-09 19:00:00.000 +0200    26.016667
2016-09-09 20:00:00.000 +0200    24.583333
2016-09-09 21:00:00.000 +0200    22.038889
2016-09-09 22:00:00.000 +0200    21.522222
2016-09-09 23:00:00.000 +0200    20.438889
Name: Temperature (C), Length: 96453, dtype: float64
```



Temperature Forecasting

❖ Step 6: Implement model

```
class WeatherForecaster(nn.Module):  
    def __init__(  
        self, embedding_dim, hidden_size,  
        n_layers, dropout_prob  
    ):  
        super(WeatherForecaster, self).__init__()  
        self.rnn = nn.RNN(  
            embedding_dim, hidden_size,  
            n_layers, batch_first=True  
        )  
        self.norm = nn.LayerNorm(hidden_size)  
  
        self.dropout = nn.Dropout(dropout_prob)  
        self.fc = nn.Linear(hidden_size, 1)  
  
    def forward(self, x):  
        x, hn = self.rnn(x)  
        x = x[:, -1, :]  
        x = self.norm(x)  
        x = self.dropout(x)  
        x = self.fc(x)  
  
        return x
```

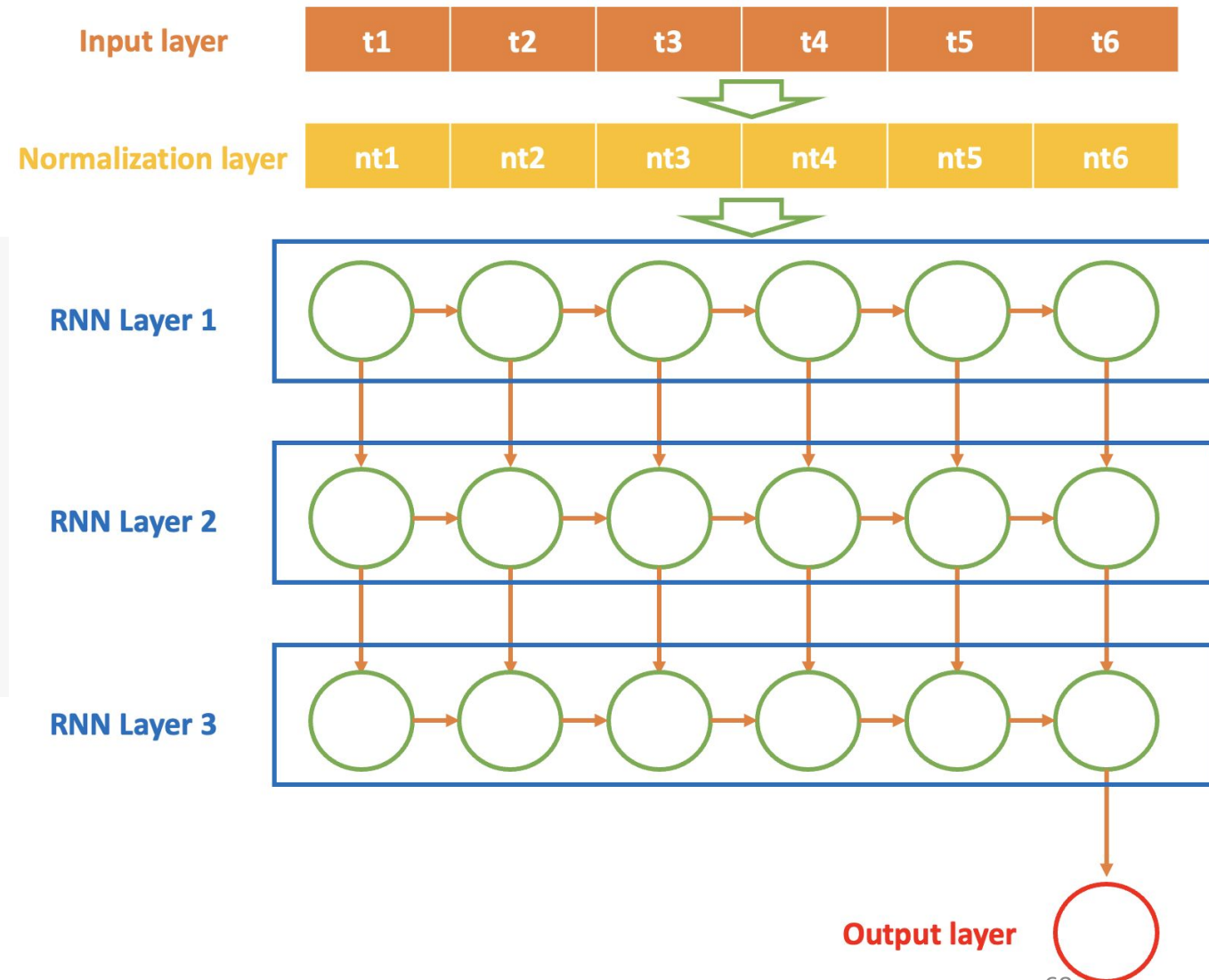


Temperature Forecasting

❖ Step 6: Implement model

```
embedding_dim = 1
hidden_size = 8
n_layers = 3
dropout_prob = 0.2
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = WeatherForecaster(
    embedding_dim=embedding_dim,
    hidden_size=hidden_size,
    n_layers=n_layers,
    dropout_prob=dropout_prob
).to(device)
```



Temperature Forecasting

◆ Step 7: Training

```
lr = 1e-3
epochs = 50

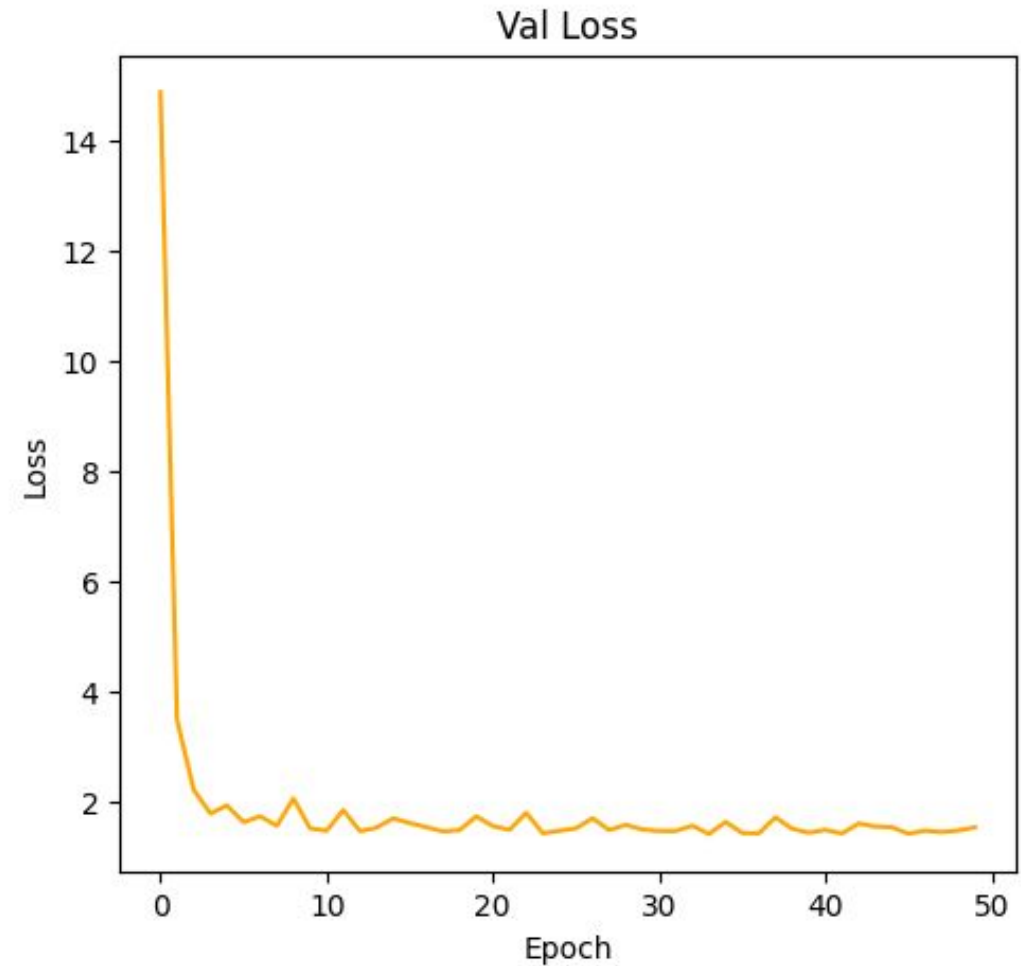
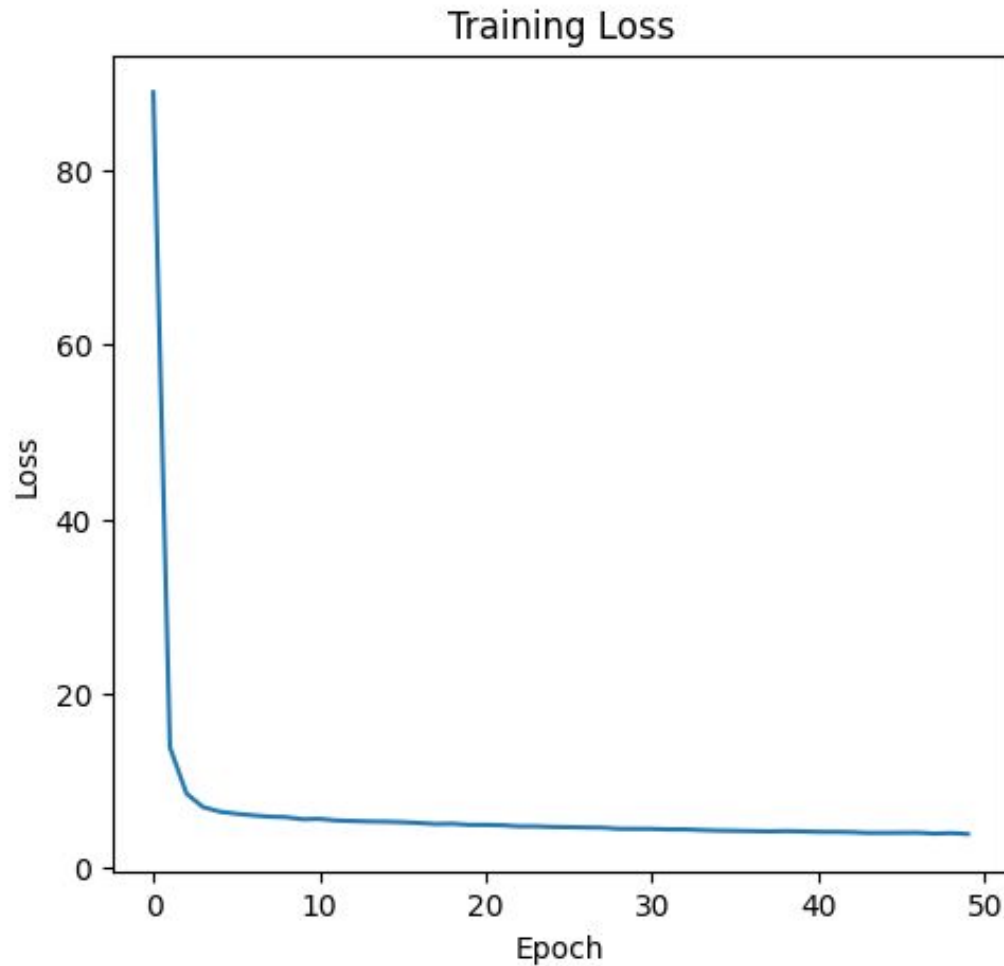
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=lr
)
```

```
train_losses, val_losses = fit(
    model,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    device,
    epochs
)
```

EPOCH 1:	Train loss: 115.3322	Val loss: 36.9400
EPOCH 2:	Train loss: 28.2214	Val loss: 7.4376
EPOCH 3:	Train loss: 15.5459	Val loss: 3.7777
EPOCH 4:	Train loss: 12.7104	Val loss: 2.5091
EPOCH 5:	Train loss: 11.1511	Val loss: 2.1068
EPOCH 6:	Train loss: 10.3305	Val loss: 1.8554
EPOCH 7:	Train loss: 9.7838	Val loss: 2.1317
EPOCH 8:	Train loss: 9.4323	Val loss: 2.2324
EPOCH 9:	Train loss: 9.2821	Val loss: 2.2774
EPOCH 10:	Train loss: 8.9802	Val loss: 1.6209
EPOCH 11:	Train loss: 8.9008	Val loss: 1.6559
EPOCH 12:	Train loss: 8.6009	Val loss: 1.6271
EPOCH 13:	Train loss: 8.4376	Val loss: 1.8376
EPOCH 14:	Train loss: 8.1859	Val loss: 2.0278
EPOCH 15:	Train loss: 8.1431	Val loss: 1.7778
EPOCH 16:	Train loss: 7.9832	Val loss: 1.6998
EPOCH 17:	Train loss: 7.6392	Val loss: 1.7180
EPOCH 18:	Train loss: 7.6296	Val loss: 1.5694

Temperature Forecasting

◆ Step 7: Training



Temperature Forecasting

◆ Step 8: Evaluation

```
1 def evaluate(model, dataloader, criterion, device):
2     model.eval()
3     correct = 0
4     total = 0
5     losses = []
6     with torch.no_grad():
7         for inputs, labels in dataloader:
8             inputs, labels = inputs.to(device), labels.to(device)
9             outputs = model(inputs)
10            loss = criterion(outputs, labels)
11            losses.append(loss.item())
12            _, predicted = torch.max(outputs.data, 1)
13            total += labels.size(0)
14            correct += (predicted == labels).sum().item()
15
16    loss = sum(losses) / len(losses)
17    acc = correct / total
18
19    return loss, acc
```

```
val_loss = evaluate(
    model,
    val_loader,
    criterion,
    device
)
test_loss = evaluate(
    model,
    test_loader,
    criterion,
    device
)

print('Evaluation on val/test dataset')
print('Val loss: ', val_loss)
print('Test loss: ', test_loss)
```

```
Evaluation on val/test dataset
Val loss:  1.5470531339433575
Test loss: 1.3722856960911496
```


Hourly Temperature Forecasting

Example: 24.86, 22.75, 20.07, 17.81, 17.16, 15.01 Target: 14.47

Enter 6 temperatures separated by commas:

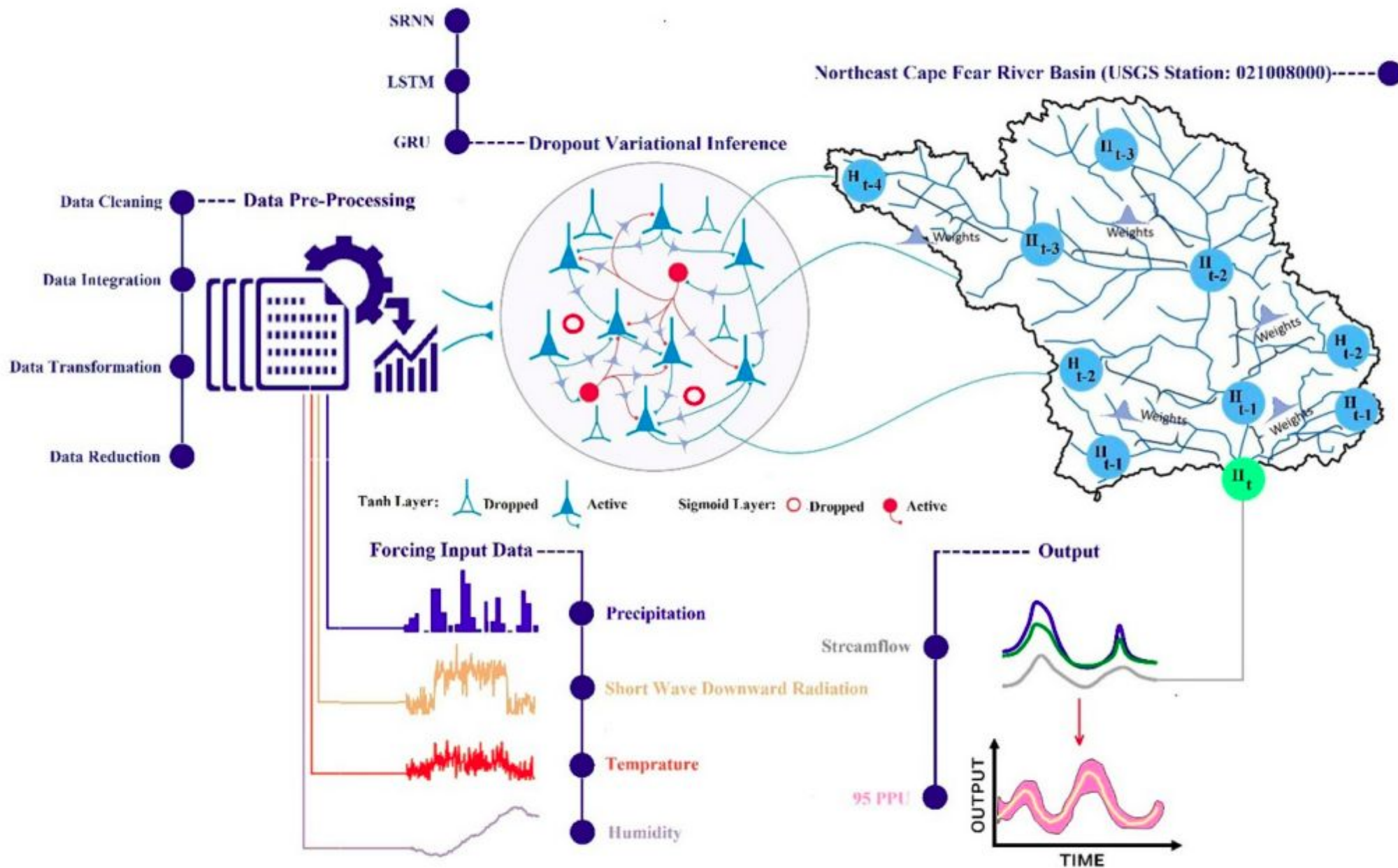
24.86, 22.75, 20.07, 17.81, 17.16, 15.01|

Predict

Predicted Temperature for the Next Hour:

Predicted Temperature: 13.67°C





Tabas, S., & Samadi, S. (2022). Variational Bayesian dropout with a Gaussian prior for recurrent neural networks application in rainfall-runoff modeling. *Environmental Research Letters*, 17. <https://doi.org/10.1088/1748-9326/ac7247>.

Figure 2. The proposed RNN workflow and structure for daily streamflow simulation across the Northeast Cape Fear Basin.

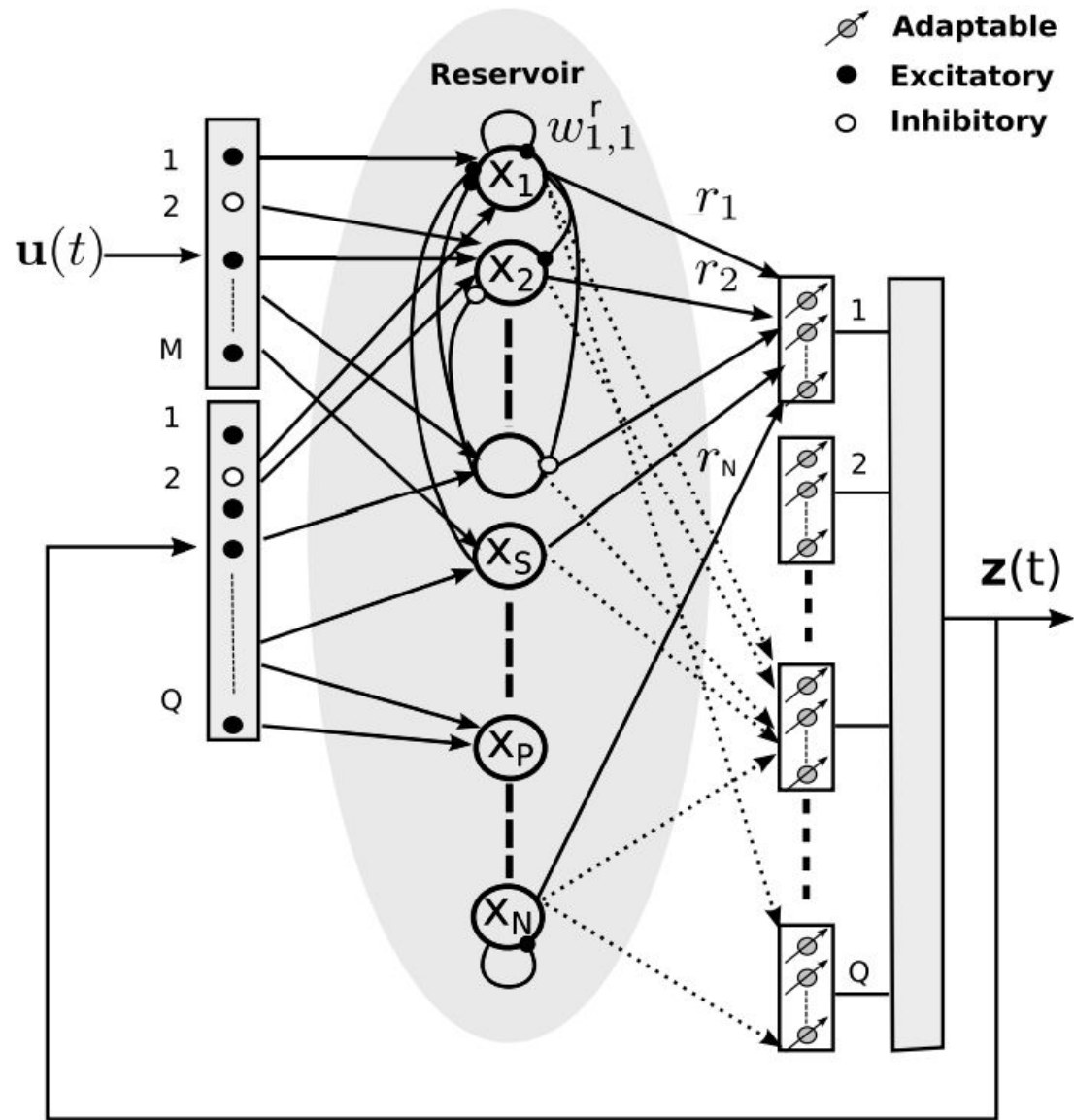


FIGURE 5. The RNN architecture consists of a reservoir of N neurons sparsely and randomly connected by excitatory and inhibitory synapses, black and white dots respectively.

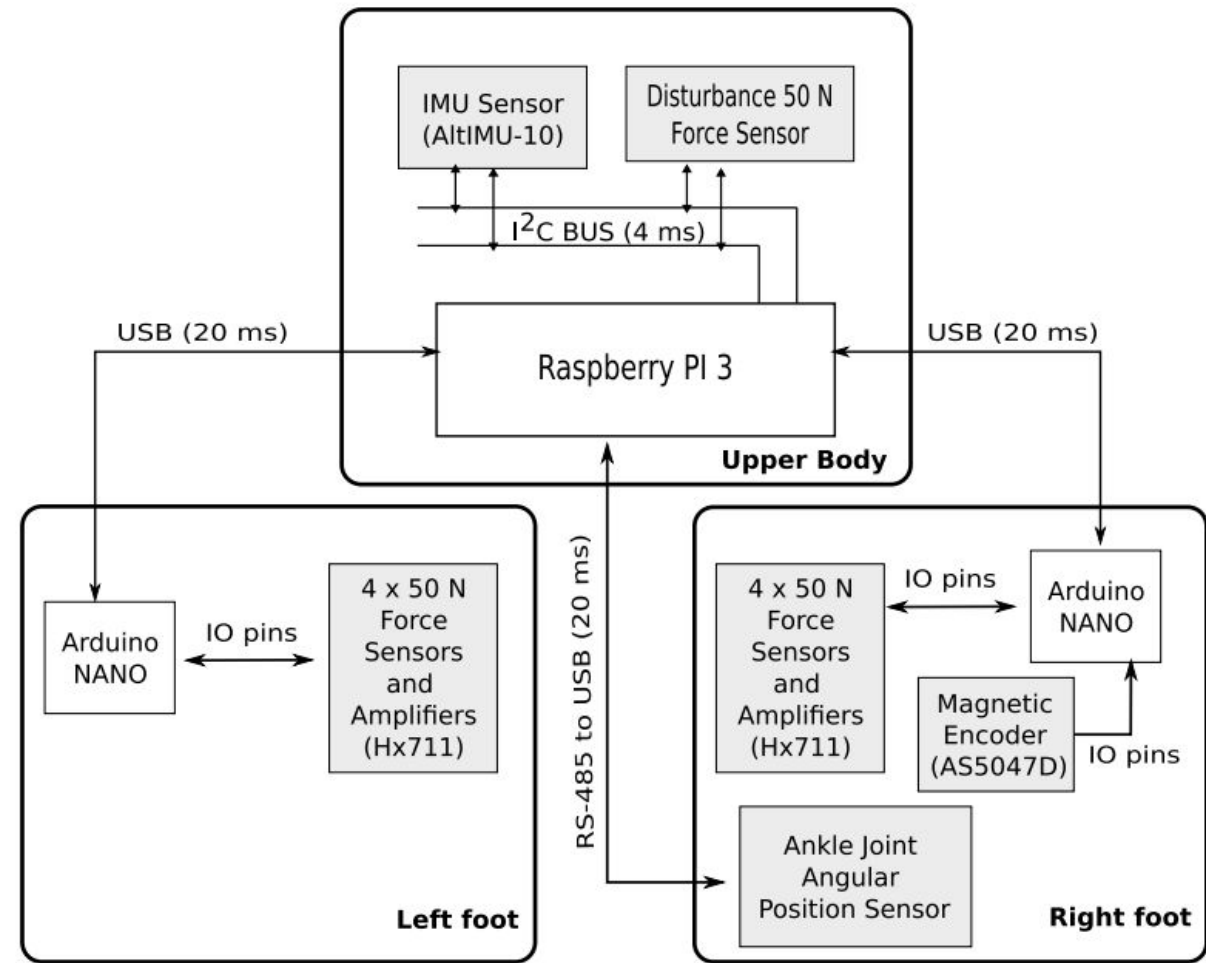


FIGURE 2. Communication protocols used to acquire the sensory data.

M. Folgheraiter, A. Yskak and S. Yessirkepov, "One-Shot Bipedal Robot Dynamics Identification With a Reservoir-Based RNN," in IEEE Access, vol. 11, pp. 50180-50194, 2023, doi: 10.1109/ACCESS.2023.3277977.,

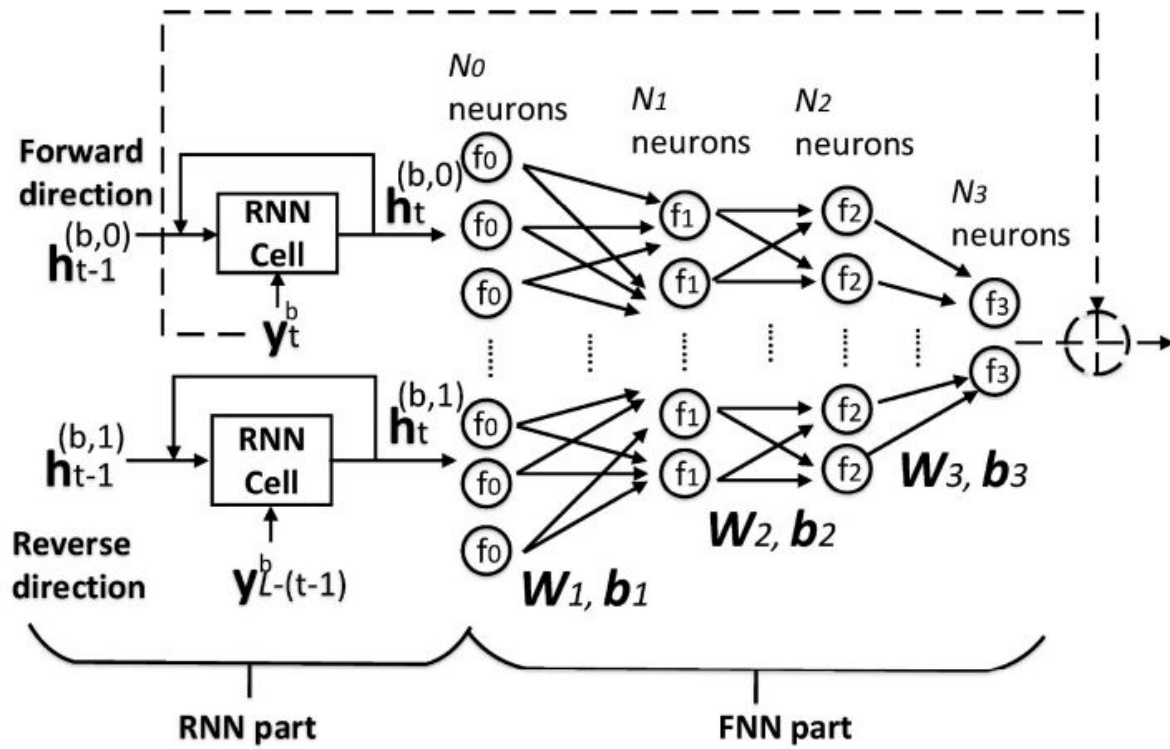


Fig. 2. A general RNN model used to describe the state-of-the-art BiLSTM model [1] and the proposed residual RNN models. The dash line is only used for the proposed residual learning method.

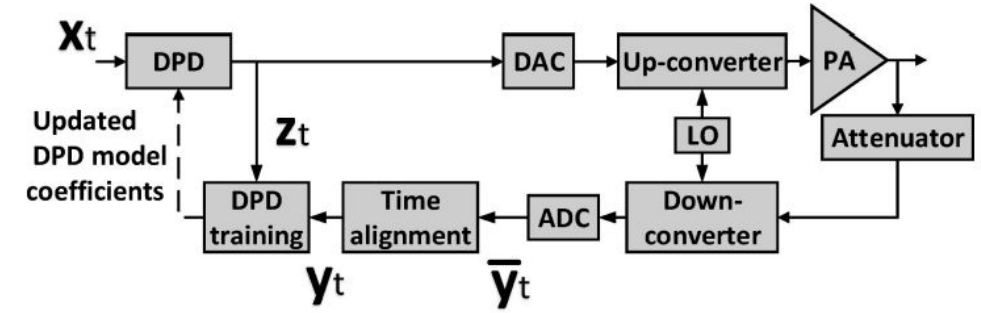


Fig. 1. A DPD system using indirect learning architecture (ILA) in a radio transmitter [3], [9].

Step-1: Initialize the model coefficients W and b

Step-2: Use y_t and z_t from multiple cycles to train the model coefficients until convergence

for $i = 1 : \mathcal{I}_{prune}$

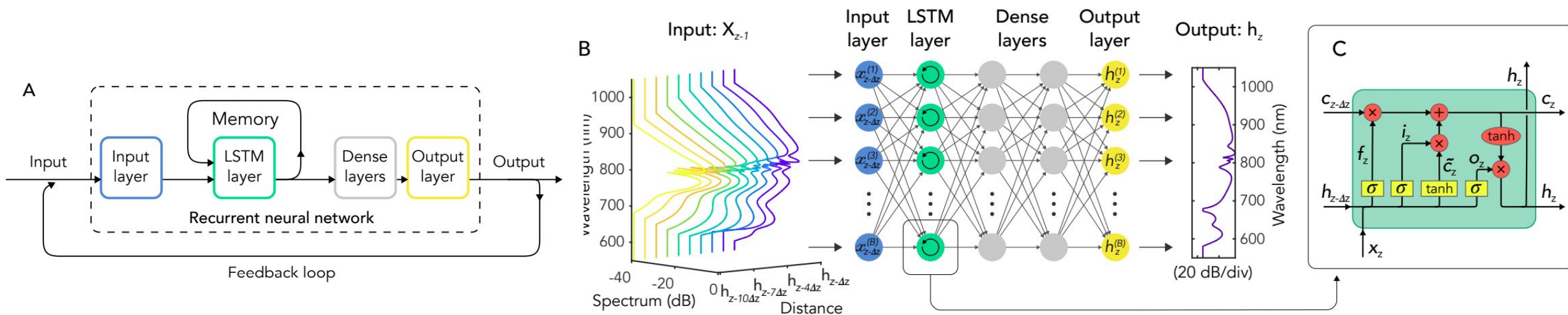
Step-3: Prune the weighting coefficients $W = \Phi_i \odot W$

Step-4: Use y_t and z_t from multiple cycles to train the un-pruned coefficients until convergence

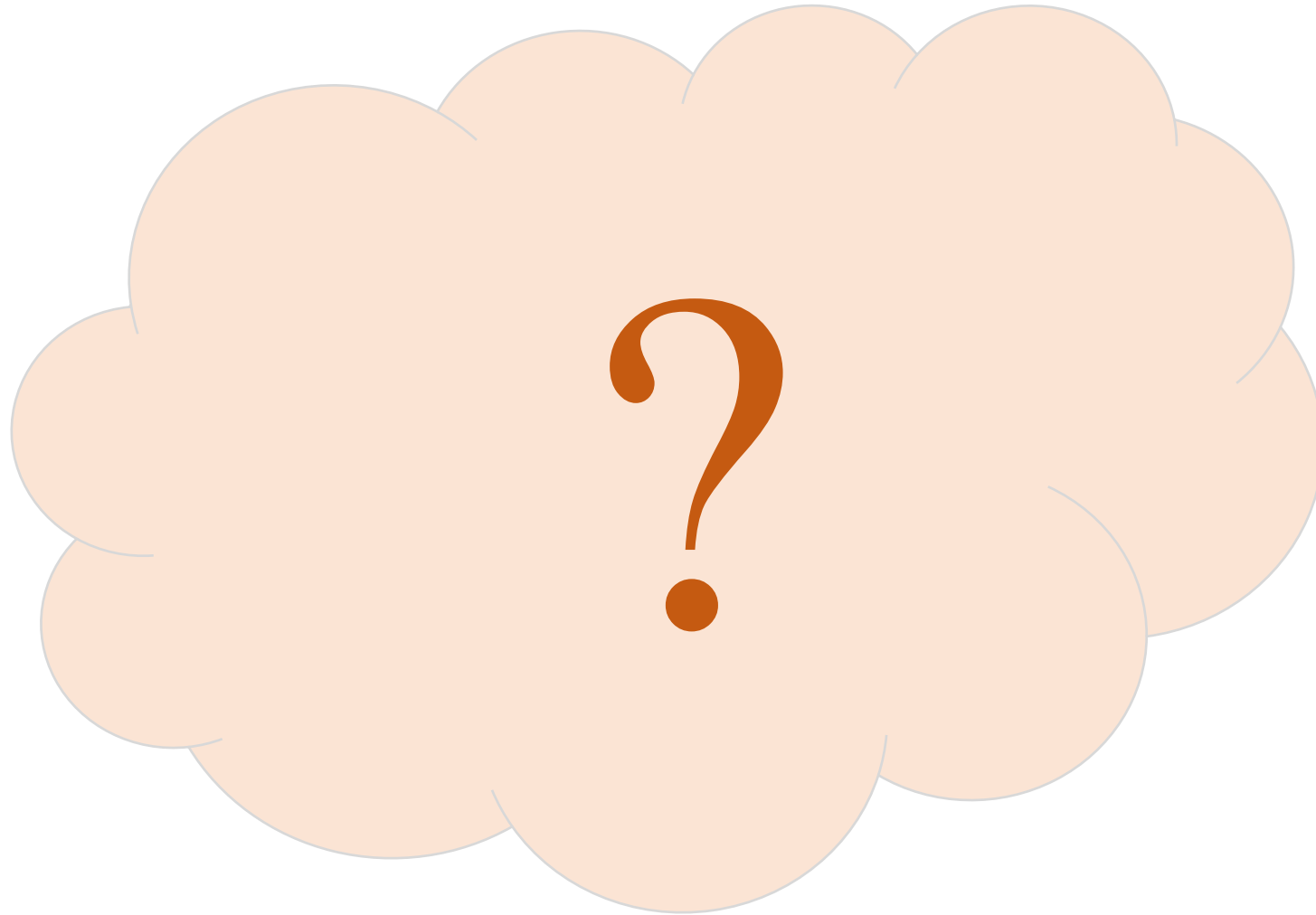
end

Objectives

- Review RNN, LSTM and Bi-LSTM architecture.
- Apply RNN, LSTM, and Bi-LSTM to Financial News Sentiment Analysis
- Apply RNN, LSTM, and Bi-LSTM to Temperature Forecasting
- Recent Papers



Question





Thank
you